

FRAMEWORKS LOT16

Autor: Jorge Serrano Blanco



indra

1. SPRING FRAMEWORK

1. CONCEPTOS BÁSICOS

La herencia en java

- Java permite el empleo de la herencia, característica muy potente que permite definir una clase tomando como base a otra clase ya existente. Esto es una de las bases de la reutilización de código, en lugar de copiar y pegar.
- En java, la herencia se especifica agregando la cláusula `extends` después del nombre de la clase. En la cláusula `extends` indicaremos el nombre de la clase base de la cuál queremos heredar.
- Al heredar de una clase base heredaremos tanto los atributos como los métodos, mientras que los constructores son utilizados, pero no heredados.

1. CONCEPTOS BÁSICOS

La herencia en java

- Java permite el empleo de la herencia, característica muy potente que permite definir una clase tomando como base a otra clase ya existente. Esto es una de las bases de la reutilización de código, en lugar de copiar y pegar.
- En java, la herencia se especifica agregando la cláusula `extends` después del nombre de la clase. En la cláusula `extends` indicaremos el nombre de la clase base de la cuál queremos heredar.
- Al heredar de una clase base heredaremos tanto los atributos como los métodos, mientras que los constructores son utilizados, pero no heredados.

1. CONCEPTOS BÁSICOS

Sobrecarga

- En Java, dos o más métodos dentro de la misma clase pueden compartir el mismo nombre, siempre que sus declaraciones de parámetros sean diferentes.
- Cuando este es el caso, se dice que los **métodos están sobrecargados**, y el proceso se denomina sobrecarga de métodos (method overloading). La sobrecarga de métodos es una de las formas en que Java implementa **polimorfismo**.

1. CONCEPTOS BÁSICOS

Interface e implementación

- Una interfaz es como Java Class, pero solo tiene constantes estáticas y método abstracto. Java usa la interfaz para implementar herencia múltiple. Una clase Java puede implementar múltiples interfaces Java. Todos los métodos en una interfaz son implícitamente públicos y abstractos.
- Un interfaz se utiliza definiendo una clase que *implemente* el interfaz a través de su nombre. Cuando una clase implementa un interfaz, debe proporcionar la definición completa de todos los métodos declarados en el interfaz y, también, la de todos los métodos declarados en todos los superinterfaces de ese interfaz.

1. CONCEPTOS BÁSICOS

Interface e implementación

- Una clase puede implementar más de un interfaz, incluyendo varios nombre de interfaces separados por comas. En este caso, la clase debe proporcionar la definición completa de todos los métodos declarados en todos los interfaces de la lista y de todos los superinterfaces de esos interfaces.

1. SPRING FRAMEWORK: DEFINICIÓN

Spring (<https://spring.io/>) es un framework de código abierto para la creación de aplicaciones empresariales Java. Tiene una estructura modular y gran flexibilidad para implementar diferentes tipos de arquitectura según las necesidades de la aplicación.



<https://spring.io/>

1. SPRING FRAMEWORK: DEFINICIÓN

- Un **Framework** es un **conjunto de artefactos software, de conceptos y de metodologías**, que a su vez tienen un conjunto de librerías, de clases, de funciones y de utilidades que nos permiten realizar algunos procesos. Un buen ejemplo de librería sería una librería matemática, al cual le podemos dar muchos datos y nos puede calcular, por ejemplo, la desviación típica
- Nos provee de un **mecanismo genérico para resolver uno o más problemas** de un tipo determinado
- **Es extensible** a través de código escrito por los usuarios
- Ofrece **facilidad para el desarrollo y despliegue**
- Si tuviéramos que desarrollar una aplicación web, podríamos **utilizar un framework que nos facilite la tarea, que nos aporte soluciones a ese desarrollo.**

1. SPRING FRAMEWORK: DEFINICIÓN

Una **aplicación empresarial** normalmente es:

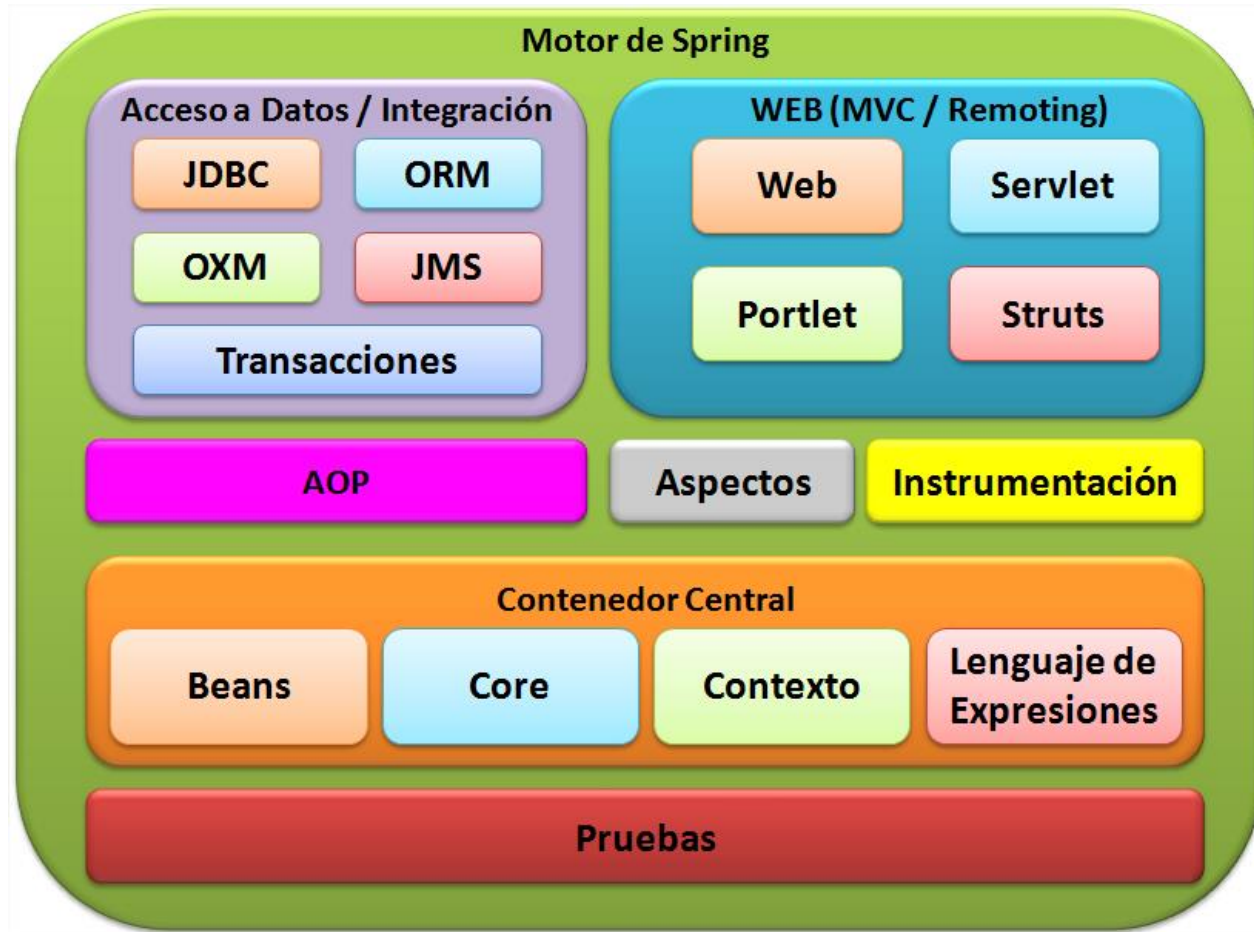
- Una gran aplicación, es decir, suele ser bastante amplia y **orientada a un ámbito comercial o industrial**
- **Compleja**, es decir, con bastantes funcionalidades, con muchos requisitos y que será utilizada por muchas personas, por lo que debería ser escalable, que pueda crecer con el tiempo. Es decir, que si hoy tenemos mil usuarios y dentro de un año tenemos un millón, que no haya que rehacer la aplicación de nuevo
- **Distribuida**, es decir, que incluso la podamos tener deslocalizada en distintos servidores
- Crítica, es decir, que **no permita o no tolere fallos** e incluso, si tuviéramos algún tipo de fallo que sea capaz de reponerse
- **Orientada a desplegarse dentro de redes** corporativas o, el esquema más usual a día de hoy, en internet a través de la nube
- **Centrada en los datos**, por lo que todas las funcionalidades se suelen desarrollar en torno a los mismos
- **Intuitiva**, de un uso fácil para evitar el rechazo de los usuarios
- Suele tener, además, unos **grandes requisitos de seguridad y de mantenibilidad**

1. SPRING FRAMEWORK: DEFINICIÓN

Spring tiene **estructura modular**:

- Cada uno de estos **módulos** se encarga de partes **diferentes** de nuestra aplicación, pudiendo **ahorrarnos mucho trabajo** ya que puede coordinar todas las partes de la aplicación
- Esta separación en módulos nos permite **usar solo las partes que necesitamos** sin tener la carga de las que no usemos
- Si utilizamos Spring de la forma correcta, nuestra aplicación quedará **dividida en capas bien delimitadas**, y con **buenas prácticas de programación**

1. SPRING FRAMEWORK: DEFINICIÓN



1. SPRING FRAMEWORK: DEFINICIÓN

FAMILIA DE PROYECTOS **SPRING**



**Spring
Framework**



Spring Boot



Spring Data



**Spring
Security**



Spring Cloud



**Spring
HATEOAS**



Spring Batch



**Spring for
Android**

1. SPRING FRAMEWORK: DEFINICIÓN

Entre las **características de Spring**, tenemos las siguientes que ofrecen una cantidad considerable de servicios:

- **Tecnologías:** como la inyección de dependencias, eventos, recursos, i18n, validación, enlace de datos, conversión de tipo, SpEL
- **Acceso a datos.** Soporte DAO, JDBC, ORM, Marshalling XML
- **Gestión de transacciones**
- **Integración:** comunicación remota, JMS, JCA, JMX, correo electrónico, tareas, programación, caché
- **Pruebas (Testing):** simulacro de objetos, el framework TestContext, Spring MVC prueba, WebClient
- **Programación orientada a aspectos (AOP):** permite la implementación de rutinas transversales
- **MVC (Modelo Vista Controlador)**

1. SPRING FRAMEWORK: DEFINICIÓN

- **Seguridad**
- **Frameworks web:** Spring WebFlux y Spring MVC.
- **Procesamiento de datos por lotes**
- **Administración Remota:** a través de este módulo se puede configurar la visibilidad y gestión de los objetos Java para la configuración local o remota vía JMX
- Es un framework liviano debido a su implementación **POJO (Plain Old Java Object)**, Spring Framework no obliga al programador a heredar ninguna clase ni a implementar ninguna interfaz.

1. SPRING FRAMEWORK: DEFINICIÓN

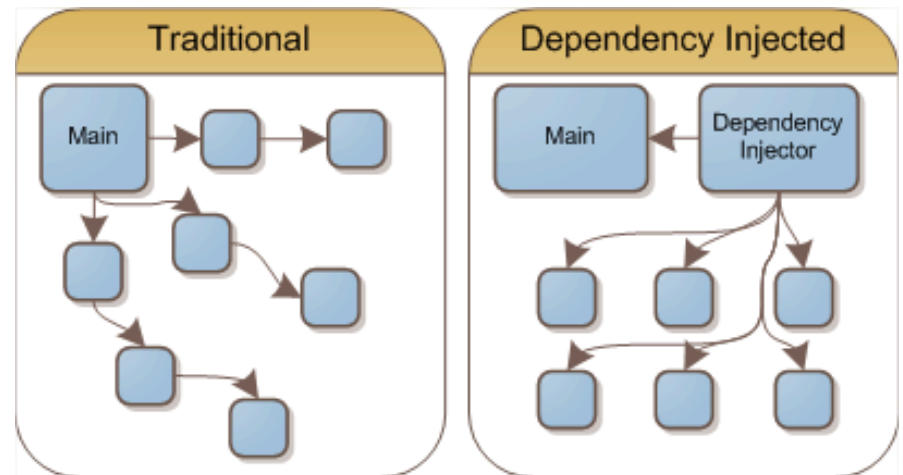
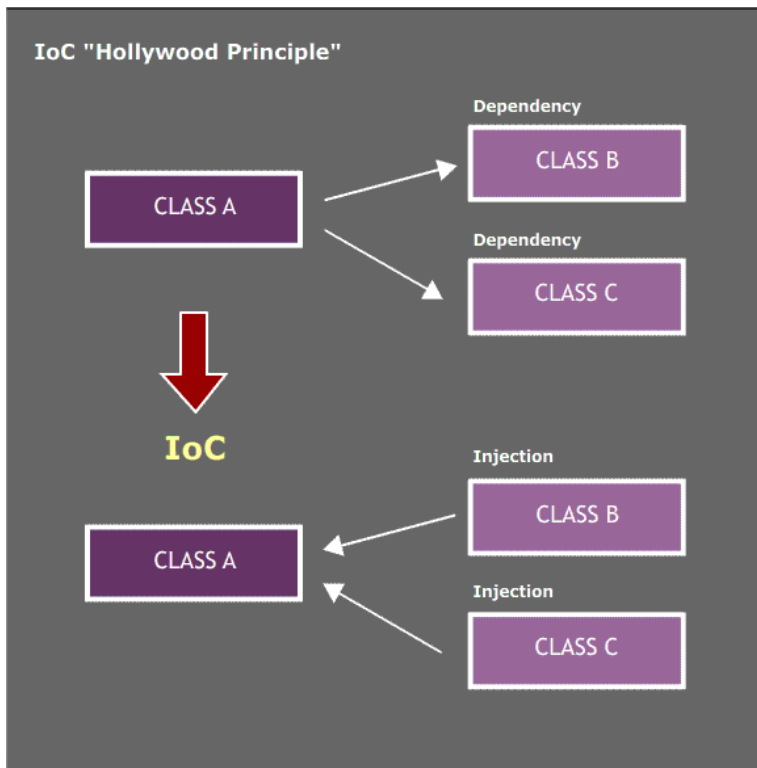
Además hay que destacar las siguientes:

- **La Inyección de Dependencias (DI – Dependency Injection):** Al momento de escribir una aplicación Java compleja, las clases de la aplicación deben de ser lo más independientes posible de otras clases Java, para aumentar la posibilidad de reutilizarlas y probarlas independientemente de otras clases, mientras se prueban las unidades. Básicamente el patrón de diseño de inyección de dependencias ayuda a unir estas clases y al mismo tiempo mantenerlas

1. SPRING FRAMEWORK: DEFINICIÓN

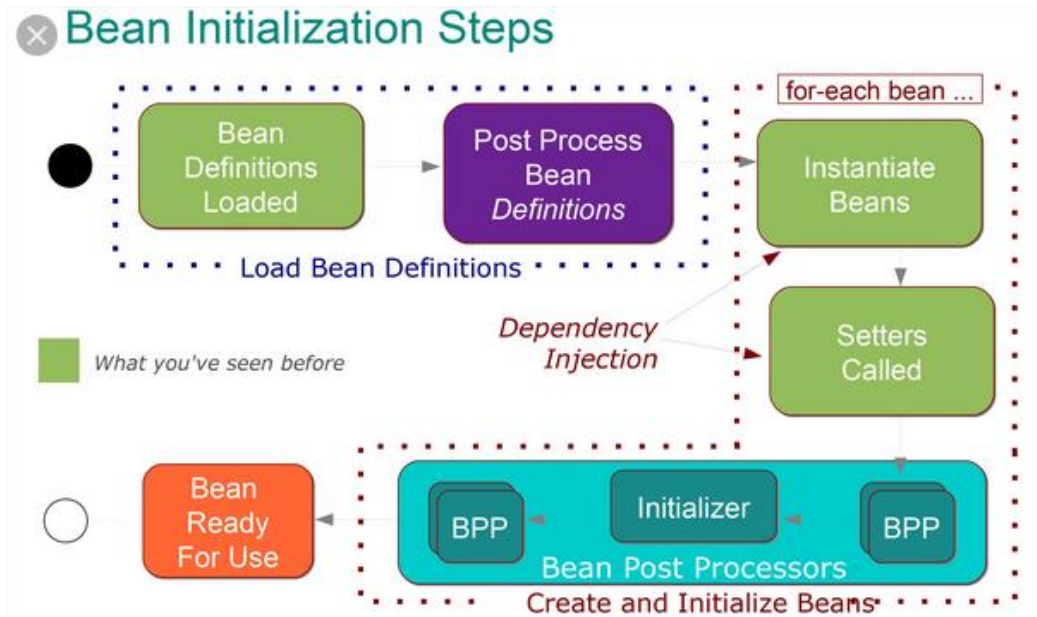
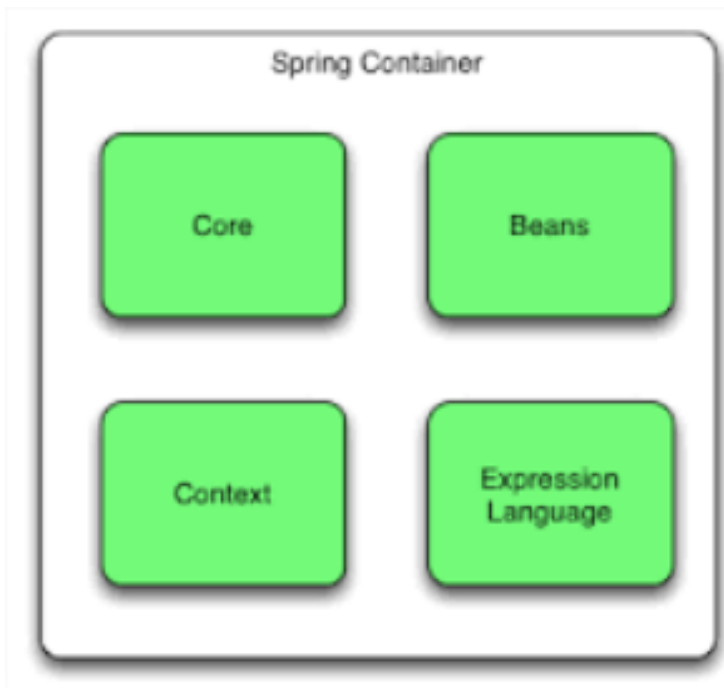
- Junto con la DI, se utiliza la **Inversión de Control (IoC)** que es un principio de diseño de software en el que el flujo de ejecución de un programa se invierte a los métodos de programación tradicionales. En los métodos de programación tradicionales la interacción se expresa de forma imperativa haciendo llamadas a procedimientos o funciones. Tradicionalmente el programador especifica la secuencia de las decisiones y procedimientos que pueden darse durante el ciclo de vida de un programa mediante llamadas a funciones. En su lugar, en la inversión de control se especifican respuestas deseadas a sucesos o solicitudes de datos concretas, dejando que algún tipo de entidad o arquitectura externa lleve a cabo las acciones de control que se requieran en el orden necesario y para el conjunto de sucesos que tengan que ocurrir

1. SPRING FRAMEWORK: DEFINICIÓN



1. SPRING FRAMEWORK: DEFINICIÓN

■ ApplicationContext y Beans de Spring



1. SPRING FRAMEWORK: DEFINICIÓN

A continuación listamos algunos de los beneficios de implementar Spring Framework:

- Spring está organizado de **forma modular**. A pesar de la cantidad de paquetes y clases que tiene, solo debemos ocuparnos de aquellos que necesitamos para nuestro desarrollo e ignorar el resto
- Utiliza alguna de las tecnologías existentes como varios frameworks **ORM**, **JEE**, temporizadores **Quartz** y **JDK**, frameworks de registro y otras tecnologías de visualización
- El framework web de Spring es un framework **MVC** web bien diseñado, que proporcionará una excelente alternativa a los frameworks web como Struts y otros frameworks web sobre diseñados o menos populares
- Spring proporciona una **API** para traducir excepciones específicas de la tecnología (como por ejemplo las generadas por JDBC, Hibernate o JDO) en excepciones consistentes y no verificadas

1. SPRING FRAMEWORK: DEFINICIÓN

- Los contenedores de **IoC (Inversion of Control)** tienden a ser livianos, especialmente cuando se comparan con los Enterprise **JavaBeans (EJB)**. Esto es ideal para desarrollar y desplegar aplicaciones en máquinas con memoria y recursos limitados
- Spring proporciona una interfaz de gestión de transacciones coherente que puede reducirse a una transacción local (utilizando una única base de datos) y ampliarse a transacciones globales
- Permite separar el registro, la auditoría, las transacciones declarativas, la seguridad, el almacenamiento en caché de la lógica comercial a través de la AOP (Programación Orientada a Aspectos)
- Cuenta con plantillas para diversas tecnologías entre las cuales podemos destacar las siguientes: JDBC, Hibernate y JPA, de tal forma que no hay necesidad de escribir un código extenso, ya que con estas plantillas simplifica el trabajo en cuanto a los pasos básicos a implementar de estas tecnologías.

1. SPRING FRAMEWORK: DEFINICIÓN

La versión actual de Spring es la 5 y se puede obtener desde su Web oficial:

<https://spring.io/projects/spring-framework>

Para más información, adjuntamos la documentación oficial de Spring:

<https://docs.spring.io/spring/docs/current/spring-framework-reference/index.html>



2. APACHE MAVEN

2. APACHE MAVEN

- **Maven** es una herramienta de software para la gestión y construcción de proyectos Java basado en un formato XML.
- Maven utiliza un **Project Object Model (POM)** para describir el proyecto de software a construir, sus dependencias de otros módulos y componentes externos, y el orden de construcción de los elementos. Viene con objetos predefinidos para realizar ciertas tareas claramente definidas, como la compilación del código y su empaquetado



2. APACHE MAVEN

- Una característica clave de Maven es que está listo para **usar en red**. El motor incluido en su núcleo puede **dinámicamente descargar plugins de un repositorio**, el mismo repositorio que provee acceso a muchas versiones de diferentes proyectos Open Source en Java, de Apache y otras organizaciones y desarrolladores.
- Una **caché local de artefactos** actúa como la primera fuente para sincronizar la salida de los proyectos a un sistema local
- Está construido usando una **arquitectura basada en plugins** que permite que utilice cualquier aplicación controlable a través de la entrada estándar
- Maven está construido alrededor de la idea de **reutilización**.
- Es un framework **configurable y altamente extensible**

2. APACHE MAVEN

Las partes del **ciclo de vida** principal de proyecto Maven son:

- 1. compile:** Genera los ficheros .class compilando los fuentes .java
- 2. test:** Ejecuta los test automáticos de Junit existentes, abortando el proceso si alguno de ellos falla
- 3. Package:** Genera el fichero .jar con los .class compilados
- 4. Install:** Copia el fichero .jar a un directorio de nuestro ordenador donde maven deja todos los .jar. De esta forma esos .jar pueden utilizarse en otros proyectos maven en el mismo ordenador
- 5. Deploy:** Copia el fichero .jar a un servidor remoto, poniéndolo disponible para cualquier proyecto maven con acceso a ese servidor remoto

2. APACHE MAVEN

Maven irá **verificando todas las fases del ciclo de vida** desde la primera hasta la del comando, ejecutando solo aquellas que no se hayan ejecutado previamente

También existen algunas **metas (goals)** que están **fuera del ciclo de vida** que pueden ser llamadas:

- **Clean:** Elimina todos los .class y .jar generados. Después de este comando se puede comenzar un compilado desde cero
- **Assembly:** Genera un fichero .zip con todo lo necesario para instalar nuestro programa java. Se debe configurar previamente en un fichero xml que se debe incluir en ese zip
- **Site:** Genera un sitio web con la información de nuestro proyecto. Dicha información debe escribirse en el fichero pom.xml y ficheros .apt separados
- **Site-deploy:** Sube el sitio web al servidor que hayamos configurado
- Etc.

2. APACHE MAVEN

Practica Spring Framework + Maven

Descargar las siguientes herramientas: JDK 1.8:

- <https://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html>

Eclipse IDE e instalar Spring Framework Plugin:

- https://www.eclipse.org/downloads/download.php?file=/technology/epp/downloads/release/2019-06/R/eclipse-jee-2019-06-R-win32-x86_64.zip
- Seguir las instrucciones de la siguiente URL
<https://spring.io/guides/gs/maven/>

3. SPRING BOOT

3. SPRING BOOT

Spring Boot es una herramienta que nace con la finalidad de simplificar aun más el desarrollo de aplicaciones basadas en el framework Spring.

Para entender el concepto primero debemos reflexionar sobre como construimos aplicaciones con Spring Framework



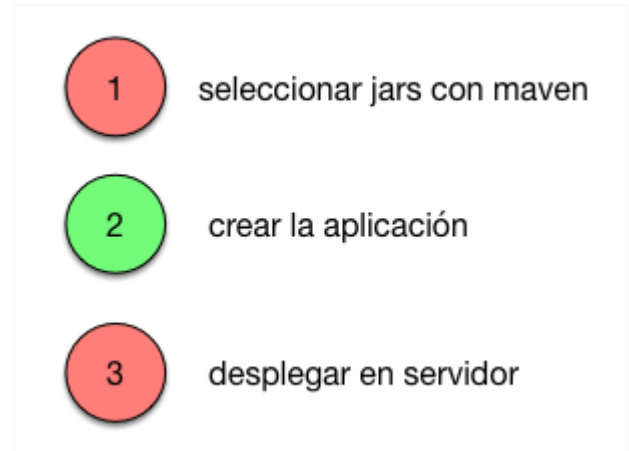
<https://spring.io/projects/spring-boot>

3. SPRING BOOT

Fundamentalmente existen 3 pasos a realizar:

1. Crear un proyecto Maven y descargar las dependencias necesarias
2. Desarrollar la aplicación
3. Desplegar la aplicación en un servidor

Únicamente el paso 2 es tarea de desarrollo.
Los otros pasos están orientados a
Infraestructura.



Spring Boot nace con la intención de simplificar los pasos 1 y 3, y que nos podamos centrar en el desarrollo de nuestra aplicación.

3. SPRING BOOT

Spring Boot centra su éxito en las siguientes características que lo hacen extremadamente fácil de utilizar:

- Configuración: Autoconfigura todos los aspectos de nuestra aplicación para poder simplemente ejecutar la aplicación, sin tener que definir absolutamente nada
- Resolución de dependencias: solo hay que determinar que tipo de proyecto estaremos utilizando y él se encarga de resolver todas las librerías/dependencias para que la aplicación funcione
- Despliegue: Se puede ejecutar como una aplicación Stand-alone, pero también es posible ejecutar aplicaciones web.
- Métricas: por defecto, cuenta con servicios que permite consultar el estado de salud de la aplicación
- Extensible: permite la creación de complementos

3. SPRING BOOT

Práctica con Spring Framework, Maven y Spring Boot

- Seguir las instrucciones de la siguiente URL

<https://spring.io/guides/gs/spring-boot/>

Práctica de ficheros de propiedades:

- <https://github.com/springframeworkguru/spring5-demo/tree/aapplication-properties>

4. SPRING MVC

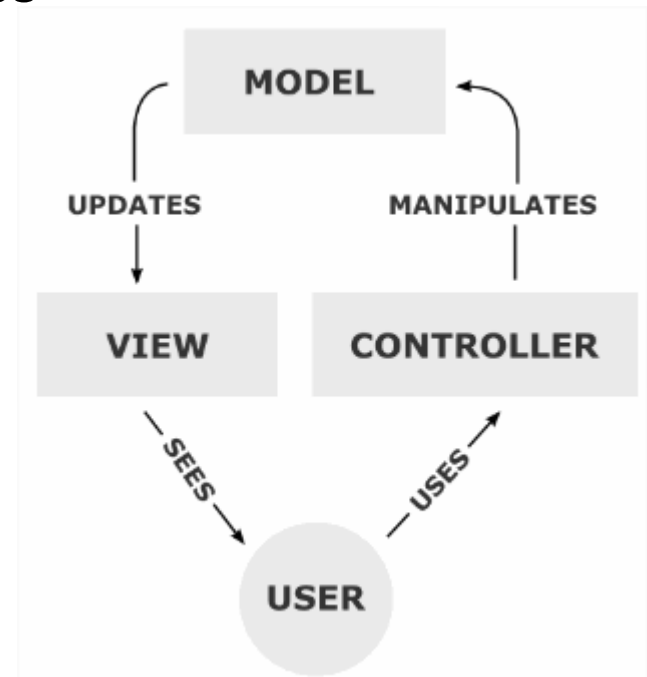
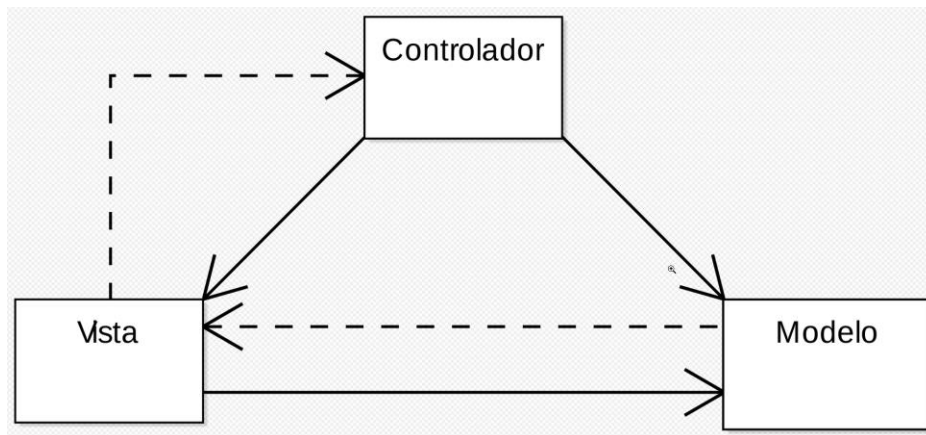
4. SPRING MVC

Spring MVC es una herramienta de Spring Framework la cual me permite aplicar el patrón de diseño sobre las aplicaciones construidas en Spring Framework



4. SPRING MVC

El **patrón de diseño MVC (Modelo-Vista-Controllador)** es un patrón de arquitectura de software, que separa los datos y la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones



4. SPRING MVC

Se dividen en 3 componentes.

- **El Modelo.** Es la representación de la información con la cual el sistema opera, gestionando todos los accesos a dicha información (consultas, actualizaciones), e implementando también los privilegios de acceso que se hayan descrito en las especificaciones de la aplicación (lógica de negocio)
- **El Controlador.** Responde a eventos (acciones del usuario) e invoca peticiones al 'modelo' cuando se hace alguna solicitud sobre la información. También puede enviar comandos a su 'vista' asociada si se solicita un cambio del 'modelo' (Middleware)
- **La Vista.** Presenta el 'modelo' en un formato adecuado para interactuar (Interfaz de usuario).

4. SPRING MVC

Ventajas:

- Lo importante de este patrón es que permite **dividir en partes**, que de alguna manera son **independientes**, con lo que si por ejemplo hago un cambio en el modelo no afectaría a la vista o si hay algún cambio sería lo mínimo

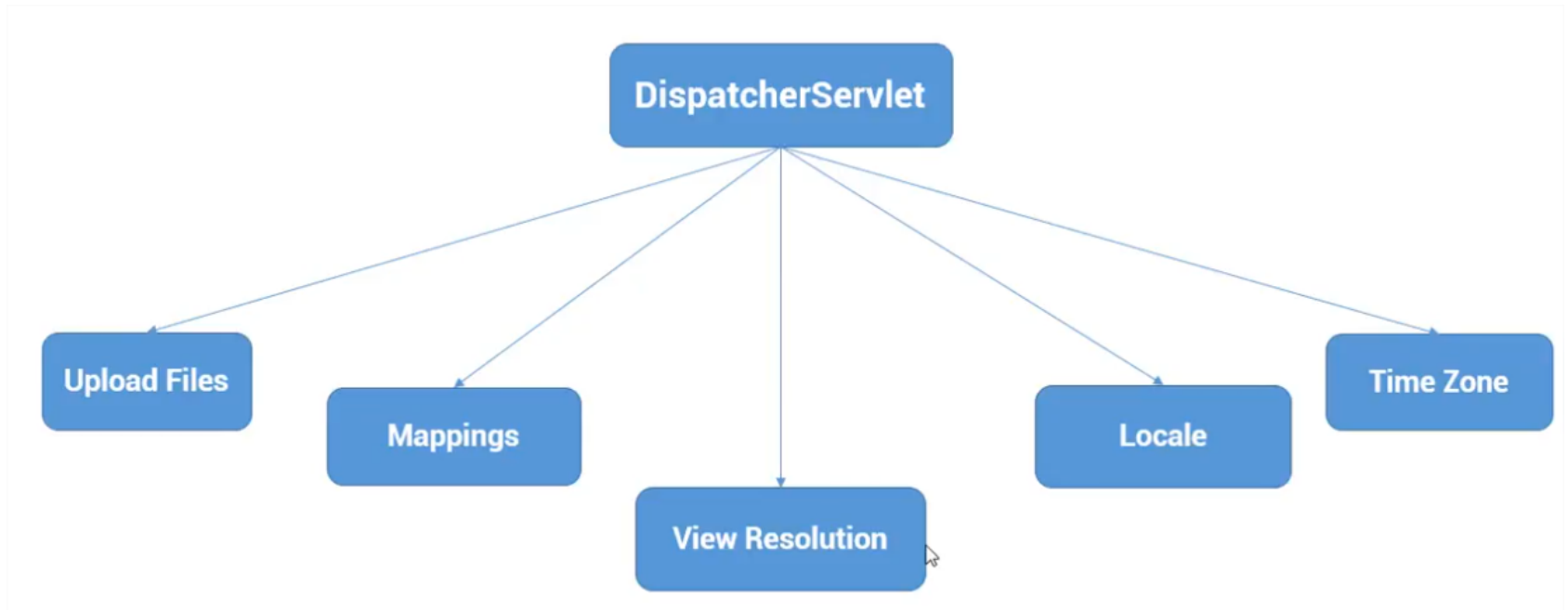


4. SPRING MVC

El DispatcherServlet es el componente principal de Spring MVC con el cual podremos:

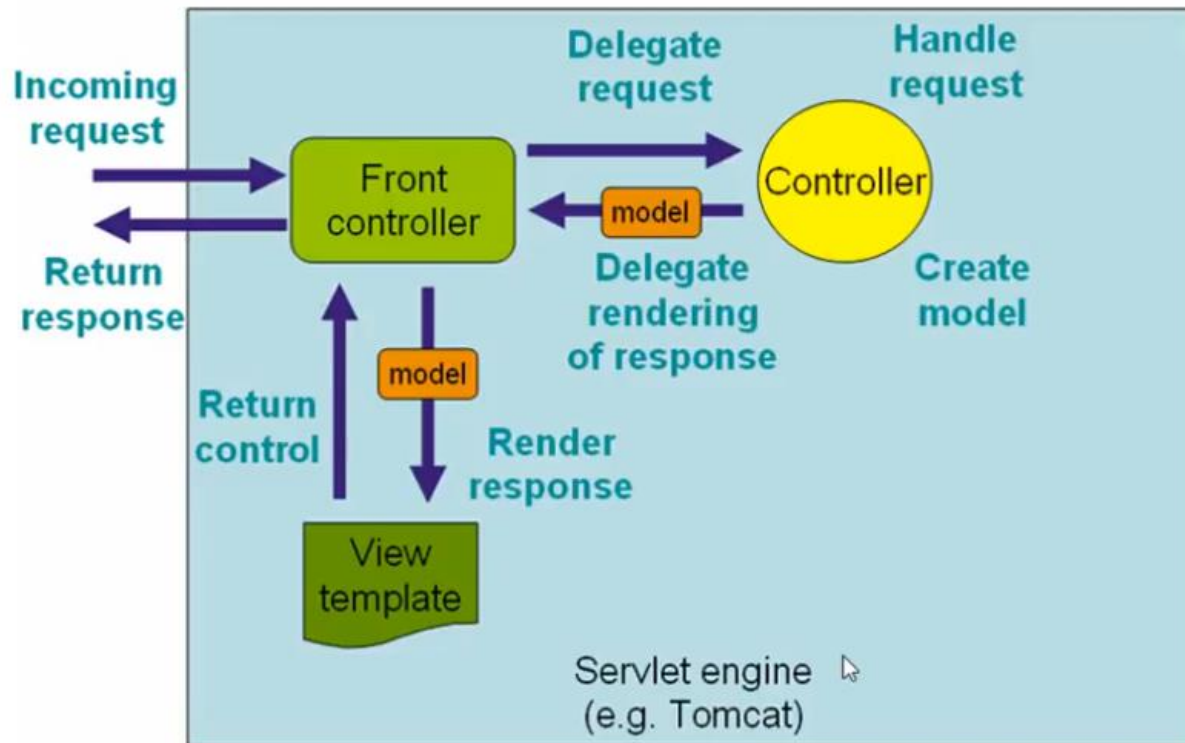
- Subir archivos
- Mapear URLs
- Renderización de vistas
- Acceder a funcionalidades de localización y zona horaria
- Etc

4. SPRING MVC



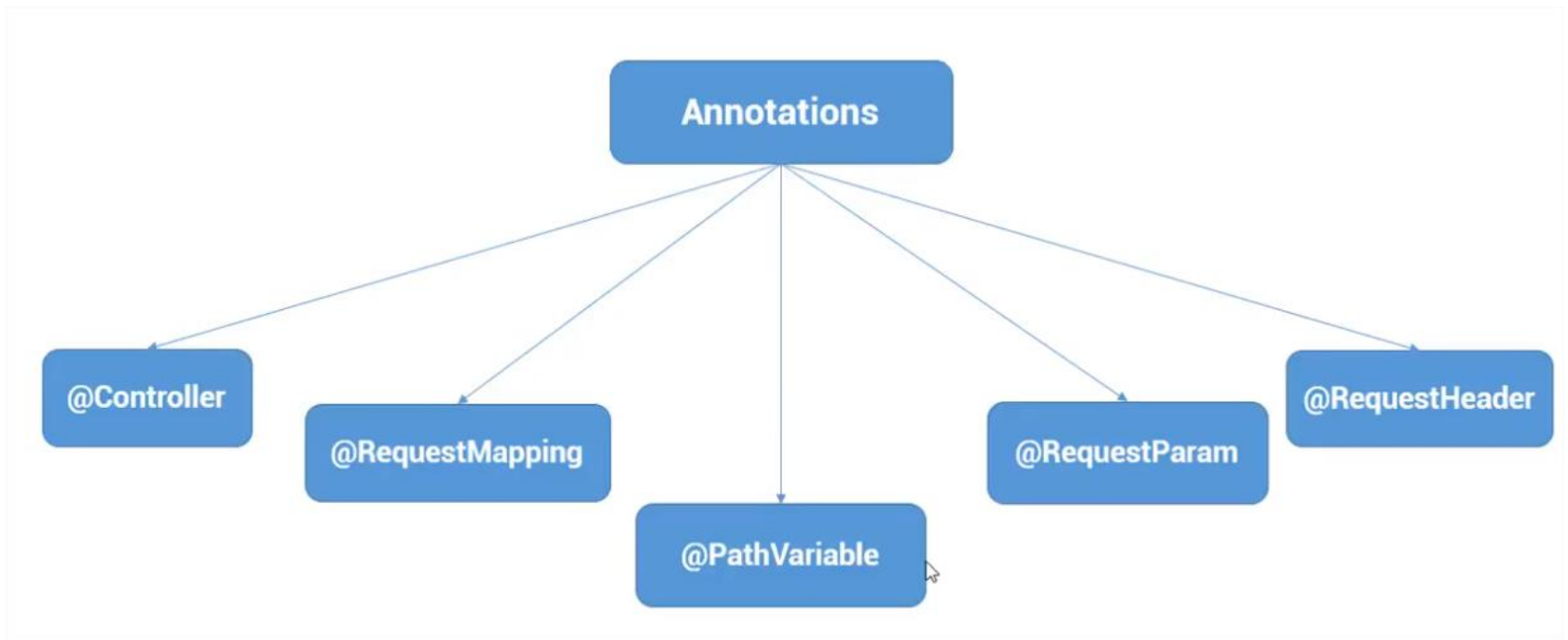
4. SPRING MVC

■ Peticiones Spring MVC



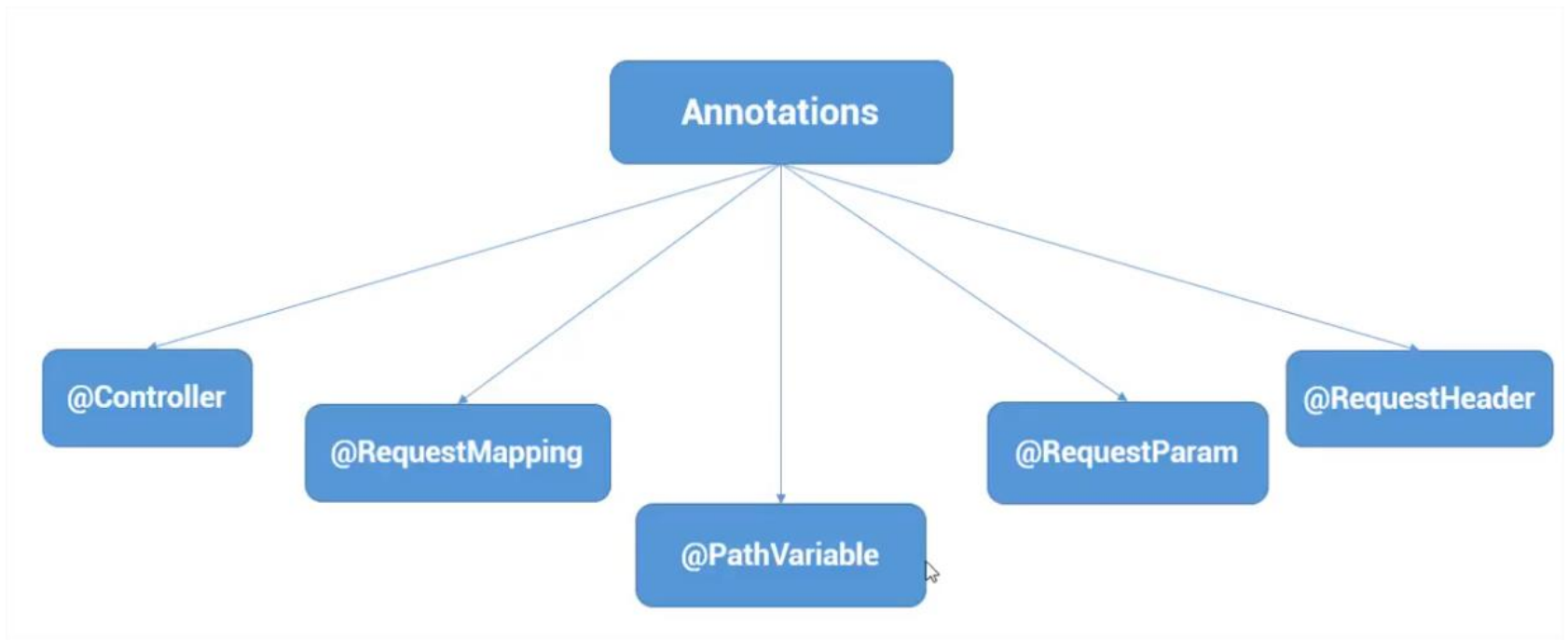
4. SPRING MVC

- Anotaciones Spring MVC



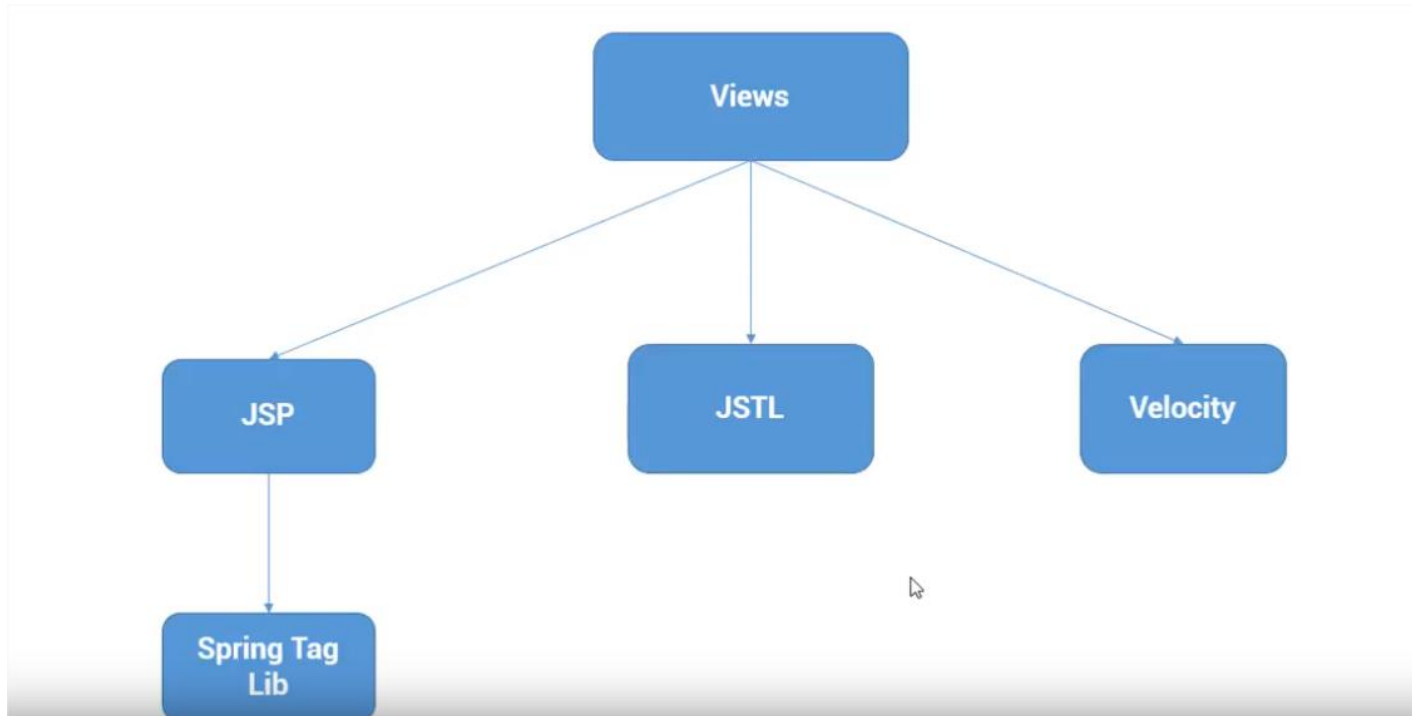
4. SPRING MVC

- Anotaciones Spring MVC



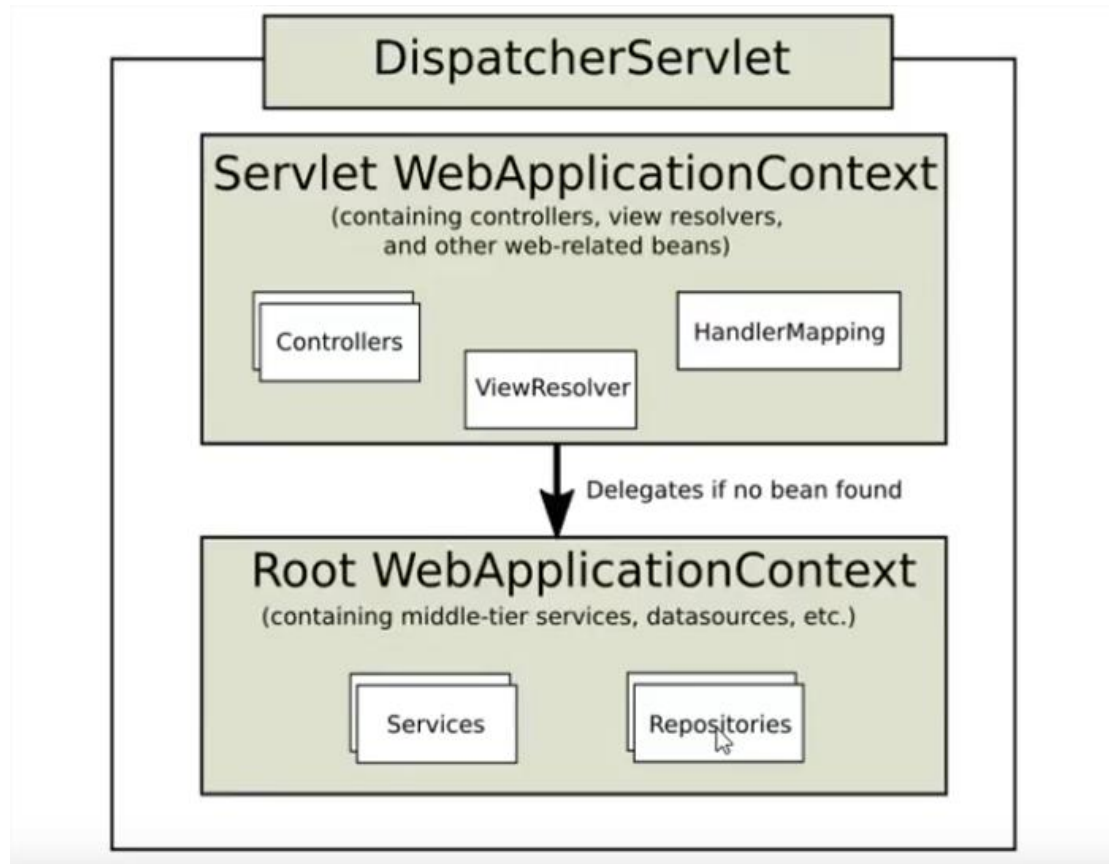
4. SPRING MVC

■ Vistas



4. SPRING MVC

■ DispatcherServlet



4. SPRING MVC

Prácticas SpringMVC:

- <https://spring.io/guides/gs/serving-web-content/>
- <https://github.com/springframeworkguru/spring5webapp/tree/display-authors>
- <https://github.com/springframeworkguru/spring5-jokes-app/tree/create-view-layer>
- <https://github.com/springframeworkguru/spring5-jokes-app/tree/xml-config-example>

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

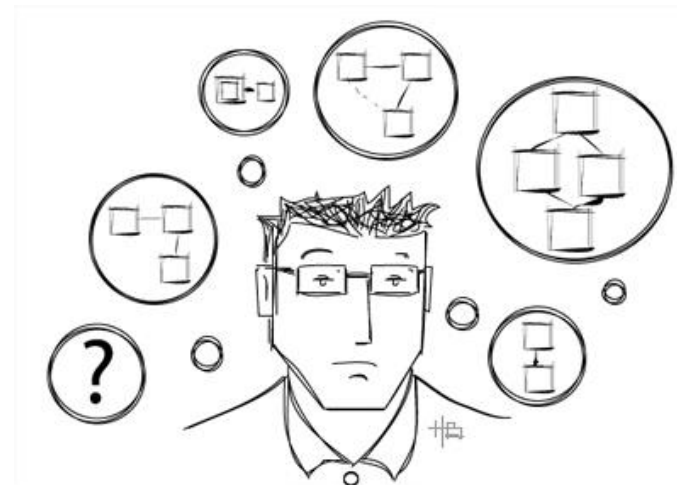
Un patrón de diseño es una solución probada que resuelve un tipo específico de problema en el desarrollo de software referente al diseño.

Existen una infinidad de patrones de diseño los mismos que se dividen en categorías. Por ejemplo:

- De creación
- Estructurales
- De comportamiento
- De interacción
- Etc.

Más información:

<http://wiki.c2.com/?DesignPatternsBook>



5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

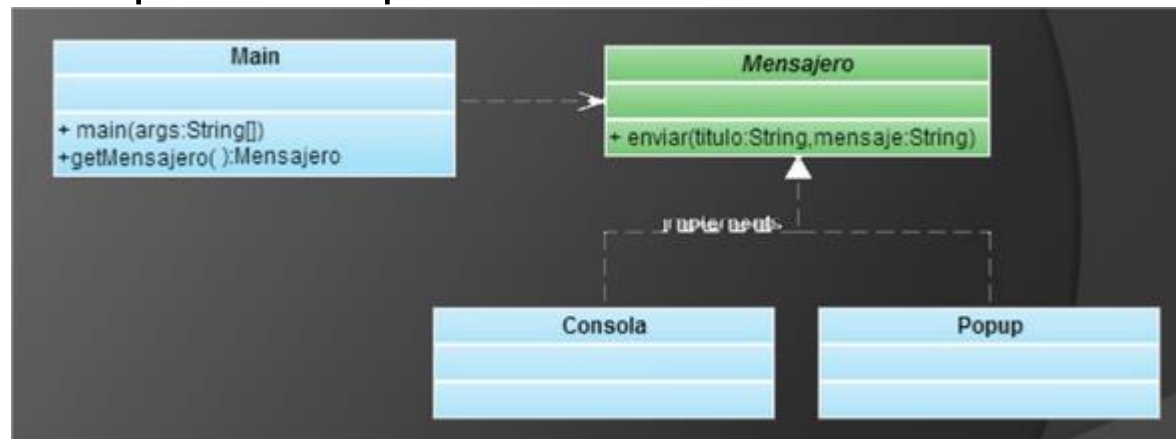
El patrón **Interface o Interfaz**:

- Es la conexión física y funcional entre dos aparatos o sistemas independientes (Polimorfismo)
- Su idea principal es separar las funciones (firma de métodos) de las implementaciones (cuerpo del método).
- Cualquier solicitud que coincida con la firma de la interfaz de un objeto también puede ser enviada a ese objeto, independientemente de su aplicación

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

Los usos más comunes para las Interfaces son:

- Deseamos describir el protocolo de comunicación de una clase fuera de su paquete
- Se debe cambiar la implementación de una funcionalidad en tiempo de ejecución
- Durante el diseño desconocemos cual será la implementación que se usará en tiempo de compilación



5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

Interfaces.java

```
1  package ingenio.ds.examples.patrones;
2
3  /*definición de la interfaz mensajero*/
4  interface Mensajero{
5      /*definición del método enviar*/
6      public void enviar(String titulo,String mensaje);
7  }
```

Clases.java

```
1  package ingenio.ds.examples.patrones;
2  import javax.swing.JOptionPane;
3
4  class Consola implements Mensajero{
5      public void enviar(String titulo,String mensaje){
6          System.out.println("Log : "+mensaje);
7      }
8  }
9
10 class Popup implements Mensajero{
11     public void enviar(String titulo,String mensaje){
12         JOptionPane.showMessageDialog(null,
13                                     mensaje,
14                                     titulo,
15                                     JOptionPane.INFORMATION_MESSAGE);
16     }
17 }
```

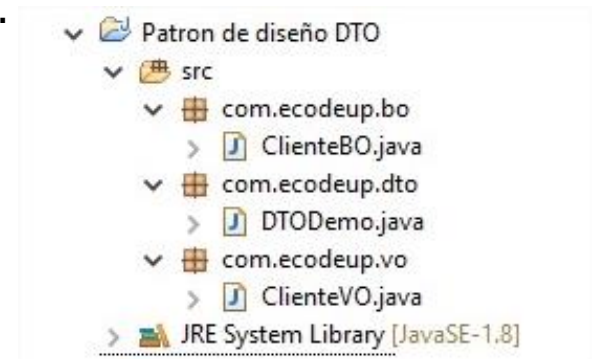
5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package ingenio.ds.examples.patrones;
2
3 public class Main {
4     /*describe el tipo de mensaje que usaremos*/
5     public static String TIPO_MENSAJE="CONSOLA";
6     public static void main(String argumentos[]){
7         /*en este punto no conocemos la implementación que se usará para enviar el
8         Mensajero mensajero=getMensajero();
9         /*llamamos el método enviar */
10        mensajero.enviar("Hola","Iniciar envío de mensaje");
11        mensajero=getMensajero();
12        mensajero.enviar("Hola","Patrón interface");
13        mensajero=getMensajero();
14        mensajero.enviar("Hola","Cerrar envío de mensaje");
15    }
16
17    public static Mensajero getMensajero(){
18        /*dependiendo del 'contexto' retornará el mensajero necesario*/
19        Mensajero mensajero=null;
20        if(TIPO_MENSAJE=="CONSOLA"){
21            mensajero=new Consola();
22            TIPO_MENSAJE="POPUP";
23        }else{
24            mensajero=new Popup();
25            TIPO_MENSAJE="CONSOLA";
26        }
27        return mensajero;
28    }
29 }
```

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

El patrón **Data Transfer Object (DTO/VO)**:

- Va de la mano con el patrón de diseño DAO
- Se utiliza para transferir varios atributos entre el cliente y el servidor o viceversa, básicamente consta de 2 clases
 - Value Object (VO) es una clase que contiene sus atributos, constructor, getters y setters (modelo)
 - Business Object (BO) es la que se encarga de obtener datos desde la base de datos, llenar el VO y enviarla al cliente, o a su vez recibir el VO desde el cliente y enviar los datos al servidor.



5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package com.ecodeup.vo;
2
3 public class ClienteVO {
4     private int id;
5     private String nombre;
6     private String apellido;
7
8
9     public ClienteVO(int id, String nombre, String apellido) {
10         this.id = id;
11         this.nombre = nombre;
12         this.apellido = apellido;
13     }
14     public int getId() {
15         return id;
16     }
17     public void setId(int id) {
18         this.id = id;
19     }
20
21     public String getNombre() {
22         return nombre;
23     }
24     public void setNombre(String nombre) {
25         this.nombre = nombre;
26     }
27
28     public String getApellido() {
29         return apellido;
30     }
31     public void setApellido(String apellido) {
32         this.apellido = apellido;
33     }
34
35     @Override
36     public String toString() {
37         return this.getNombre()+" "+this.getApellido();
38     }
39 }
```

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package com.ecodeup.bo;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import com.ecodeup.vo.ClienteVO;
7
8 public class ClienteBO {
9
10     //lista de tipo cliente
11     List<ClienteVO> clientes;
12
13
14     //constructor, se guarda en la lista 2 clientes
15     public ClienteBO() {
16         clientes = new ArrayList<>();
17         ClienteVO cliente1= new ClienteVO(0,"Elivar","Largo");
18         ClienteVO cliente2= new ClienteVO(1,"Priscila","Morochu");
19         clientes.add(cliente1);
20         clientes.add(cliente2);
21     }
22
23     //elimina el cliente que se le pasa como parámetro
24     public void eliminarCliente(ClienteVO cliente) {
25         clientes.remove(cliente.getId());
26         System.out.println("Cliente "+cliente.getId()+" eliminado satisfactoriamente");
27     }
28
29     //obtiene toda la lista de clientes
30     public List<ClienteVO> obtenerClientes(){
31         return clientes;
32     }
33
34     //obtiene un cliente de acuerdo al id pasado como parámetro
35     public ClienteVO obtenerCliente(int id) {
36         return clientes.get(id);
37     }
38
39     // actualiza el cliente que se le pasa como parámetro
40     public void actualizarCliente(ClienteVO cliente) {
41         clientes.get(cliente.getId()).setNombre(cliente.getNombre());
42         clientes.get(cliente.getId()).setApellido(cliente.getApellido());
43         System.out.println("Cliente id: " + cliente.getId()+" actualizado satisfactoriamente");
44     }
45 }
```

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
package com.ecodeup.dto;

import com.ecodeup.bo.ClienteBO;
import com.ecodeup.vo.ClienteVO;

public class DTODemo {
    public static void main(String[] args) {
        //objeto business object
        ClienteBO clienteBusinessObject = new ClienteBO();

        //obtiene todos los clientes
        clienteBusinessObject.obtenerClientes().forEach(System.out::println);

        // actualiza un cliente
        System.out.println("****");
        ClienteVO cliente = clienteBusinessObject.obtenerCliente(0);
        cliente.setNombre("Luis");
        clienteBusinessObject.actualizarCliente(cliente);

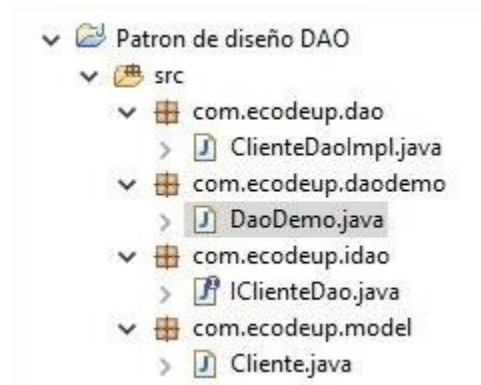
        // obtiene un cliente
        System.out.println("****");
        cliente=clienteBusinessObject.obtenerCliente(0);
        System.out.println(cliente);

        //elimina un cliente
        System.out.println("****");
        cliente=clienteBusinessObject.obtenerCliente(0);
        clienteBusinessObject.eliminarCliente(cliente);
    }
}
```


5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

El patrón **Data Acces Object (DAO)**:

- El problema que resuelve este patrón es el acceso a los datos, que básicamente tiene que ver con la gestión de diversas fuentes de datos y además abstrae la forma de acceder a ellos
- La capa DAO contiene todos los métodos CRUD (create, read, update, delete), por lo general se tiene un DAO para cada tabla en la base de datos, y bueno la implementación se realiza de la siguiente manera



5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package com.ecodeup.model;
2
3 public class Cliente {
4     private int id;
5     private String nombre;
6     private String apellido;
7
8
9     public Cliente() {
10         super();
11     }
12     public Cliente(int id, String nombre, String apellido) {
13         super();
14         this.id = id;
15         this.nombre = nombre;
16         this.apellido = apellido;
17     }
18     public int getId() {
19         return id;
20     }
21     public void setId(int id) {
22         this.id = id;
23     }
24
25     public String getNombre() {
26         return nombre;
27     }
28     public void setNombre(String nombre) {
29         this.nombre = nombre;
30     }
31
32     public String getApellido() {
33         return apellido;
34     }
35     public void setApellido(String apellido) {
36         this.apellido = apellido;
37     }
38
39     @Override
40     public String toString() {
41         return this.getNombre()+" "+this.getApellido();
42     }
43 }
```

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package com.ecodeup.idao;
2
3 import java.util.List;
4
5 import com.ecodeup.model.Cliente;
6
7 public interface IClienteDao {
8     //declaración de métodos para acceder a la base de datos
9     public List<Cliente> obtenerClientes();
10    public Cliente obtenerCliente(int id);
11    public void actualizarCliente(Cliente cliente);
12    public void eliminarCliente(Cliente cliente);
13 }
```

5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package com.ecodeup.dao;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import com.ecodeup.idao.*;
7 import com.ecodeup.model.Cliente;
8
9 public class ClienteDaoImpl implements IClienteDao {
10
11     //lista de tipo cliente
12     List<Cliente> clientes;
13
14     //inicializar los objetos cliente y añadirlos a la lista
15     public ClienteDaoImpl() {
16         clientes = new ArrayList<>();
17         Cliente cliente1 = new Cliente(0,"Javier", "Molina");
18         Cliente cliente2 = new Cliente(1,"Lillian", "Álvarez");
19         clientes.add(cliente1);
20         clientes.add(cliente2);
21     }
22
23     //obtener todos los clientes
24     @Override
25     public List<Cliente> obtenerClientes() {
26         return clientes;
27     }
28
29     //obtener un cliente por el id
30     @Override
31     public Cliente obtenerCliente(int id) {
32         return clientes.get(id);
33     }
34
35     //actualizar un cliente
36     @Override
37     public void actualizarCliente(Cliente cliente) {
38         clientes.get(cliente.getId()).setNombre(cliente.getNombre());
39         clientes.get(cliente.getId()).setApellido(cliente.getApellido());
40         System.out.println("Cliente con id: "+cliente.getId()+" actualizado satisfactoriamente");
41     }
42
43     //eliminar un cliente por el id
44     @Override
45     public void eliminarCliente(Cliente cliente) {
46         clientes.remove(cliente.getId());
47         System.out.println("Cliente con id: "+cliente.getId()+" eliminado satisfactoriamente");
48     }
49 }
```

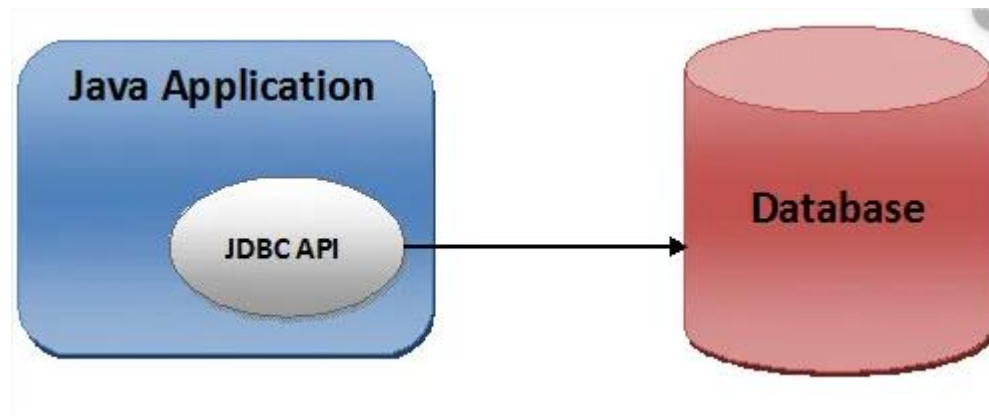
5. PATRONES DE DISEÑO INTERFACE/DTO/DAO

```
1 package com.ecodeup.daodemo;
2
3 import com.ecodeup.dao.ClienteDaoImpl;
4 import com.ecodeup.idao.IClienteDao;
5 import com.ecodeup.model.Cliente;
6
7 public class DaoDemo {
8
9     public static void main(String[] args) {
10         // objeto para manipular el dao
11         IClienteDao clienteDao = new ClienteDaoImpl();
12
13         // imprimir los clientes
14         clienteDao.obtenerClientes().forEach(System.out::println);
15
16         // obtener un cliente
17         Cliente cliente = clienteDao.obtenerCliente(0);
18         cliente.setApellido("Pardo");
19         //actualizar cliente
20         clienteDao.actualizarCliente(cliente);
21
22         // imprimir los clientes
23         System.out.println("*****");
24         clienteDao.obtenerClientes().forEach(System.out::println);
25     }
26 }
```

6. ACCESO A BASE DE DATOS: JDBC

6. ACCESO A BASE DE DATOS: JDBC

Java Database Connectivity (JDBC) es una API (Interfaz de Programación de Aplicaciones) que permite la ejecución de operaciones sobre bases de datos desde Java, independientemente del sistema operativo donde se ejecute o de la base de datos a la cual se accede, utilizando el dialecto SQL del modelo de base de datos que se utilice.



6. ACCESO A BASE DE DATOS: JDBC

- Se presenta como una **colección de interfaces Java** y métodos de **gestión de manejadores de conexión** hacia cada modelo específico de base de datos.
- Un manejador de conexiones hacia un modelo de base de datos es un conjunto de clases que implementan las interfaces Java y que utilizan los métodos de registro para declarar los tipos de localizadores a base de datos (URL) que pueden manejar
- Para poderse conectar a la Base de Datos, se necesita:
 - Localizador de BD (Máquina/IP y puerto)
 - Parámetros de conexión:
 - Usuario/Contraseña
 - Nombre de servicio/SID/Pool de conexiones

6. ACCESO A BASE DE DATOS: JDBC

- Para poder conectar a BD se necesita un **Driver JDBC** que es un adaptador del lado del cliente que convierten la petición proveniente de Java a un protocolo que el SGBD (Sistema Gestor de Base de Datos) pueda entender.
- A partir de aquí se puede realizar cualquier tipo de tarea con la base de datos a la que tenga permiso:
 - Consulta
 - Actualización
 - Creación
 - Modificación
 - Borrado
 - Ejecución de procedimientos

6. ACCESO A BASE DE DATOS: JDBC

Una simple conexión a la base de datos

```
Connection conn = null;

final String url = "jdbc:mysql://localhost:3306/"; // Nuestra cadena de conexión
final String dbName = "bookshopdb"; // Nombre de nuestra base de datos
final String driver = "com.mysql.jdbc.Driver";
final String userName = "root";
final String password = "password";

try {

    Class.forName(driver).newInstance();
    conn = DriverManager.getConnection(url + dbName, userName, password);

    if (!conn.isClosed())
        System.out.println("Database connection working using TCP/IP...");

} catch (Exception e) {

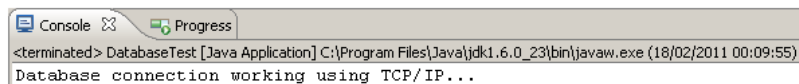
    System.err.println("Exception: " + e.getMessage());

} finally {

    try {
        if (conn != null)
            conn.close();
    } catch (SQLException e) {
    }

}
```

El resultado de su ejecución



The screenshot shows a console window with two tabs: 'Console' and 'Progress'. The 'Console' tab is active and displays the following text: '<terminated> DatabaseTest [Java Application] C:\Program Files\Java\jdk1.6.0_23\bin\javaw.exe (18/02/2011 00:09:55)' followed by 'Database connection working using TCP/IP...'. The 'Progress' tab is empty.

6. ACCESO A BASE DE DATOS: JDBC

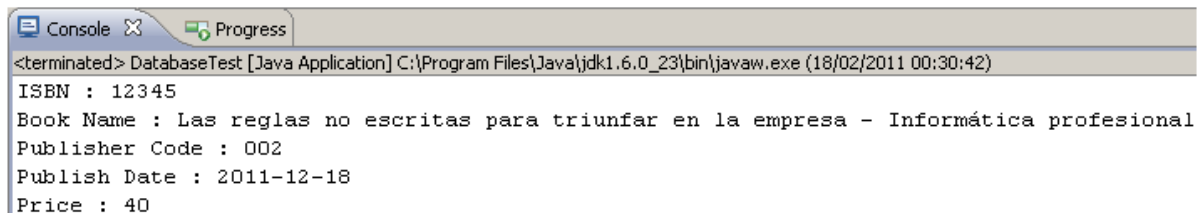
Un simple consulta

```
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM BOOK WHERE ISBN = ?");
stmt.setString(1, "12345");

ResultSet rs = stmt.executeQuery();
while (rs.next()) {
    System.out.println("ISBN : " + rs.getString("ISBN"));
    System.out.println("Book Name : " + rs.getString("BOOK_NAME"));
    System.out.println("Publisher Code : " + rs.getString("PUBLISHER_CODE"));
    System.out.println("Publish Date : " + rs.getDate("PUBLISH_DATE"));
    System.out.println("Price : " + rs.getInt("PRICE"));
    System.out.println();
}

rs.close();
stmt.close();
```

El resultado de su ejecución



The screenshot shows a Java IDE window with a 'Console' tab. The output of the program is as follows:

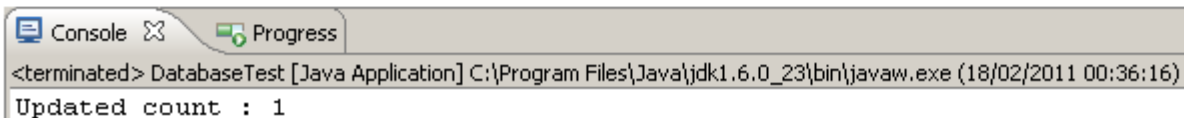
```
<terminated> DatabaseTest [Java Application] C:\Program Files\Java\jdk1.6.0_23\bin\javaw.exe (18/02/2011 00:30:42)
ISBN : 12345
Book Name : Las reglas no escritas para triunfar en la empresa - Informática profesional
Publisher Code : 002
Publish Date : 2011-12-18
Price : 40
```

6. ACCESO A BASE DE DATOS: JDBC

Una simple actualización de un registro

```
PreparedStatement stmt = conn.prepareStatement("UPDATE BOOK SET BOOK_NAME = ? WHERE ISBN = ?");  
stmt.setString(1, "2K Developer's Guide");  
stmt.setString(2, "67890");  
  
int count = stmt.executeUpdate();  
  
System.out.println("Updated count : " + count);  
  
stmt.close();
```

El resultado de su ejecución



The screenshot shows a Java IDE console window with two tabs: 'Console' and 'Progress'. The 'Console' tab is active, displaying the output of the program. The text in the console is: '<terminated> DatabaseTest [Java Application] C:\Program Files\Java\jdk1.6.0_23\bin\javaw.exe (18/02/2011 00:36:16)' followed by 'Updated count : 1'.

6. ACCESO A BASE DE DATOS: JDBC

Un simple borrado

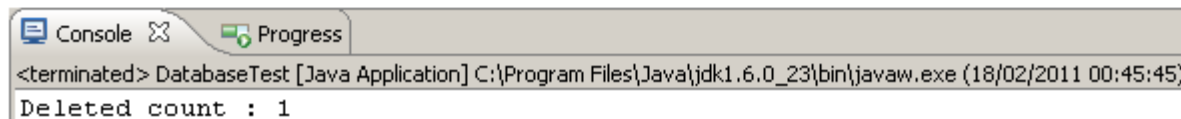
```
PreparedStatement stmt = conn.prepareStatement("DELETE FROM BOOK WHERE ISBN = ?");
stmt.setString(1, "12345");

int count = stmt.executeUpdate();

System.out.println("Deleted count : " + count);

stmt.close();
```

El resultado de su ejecución



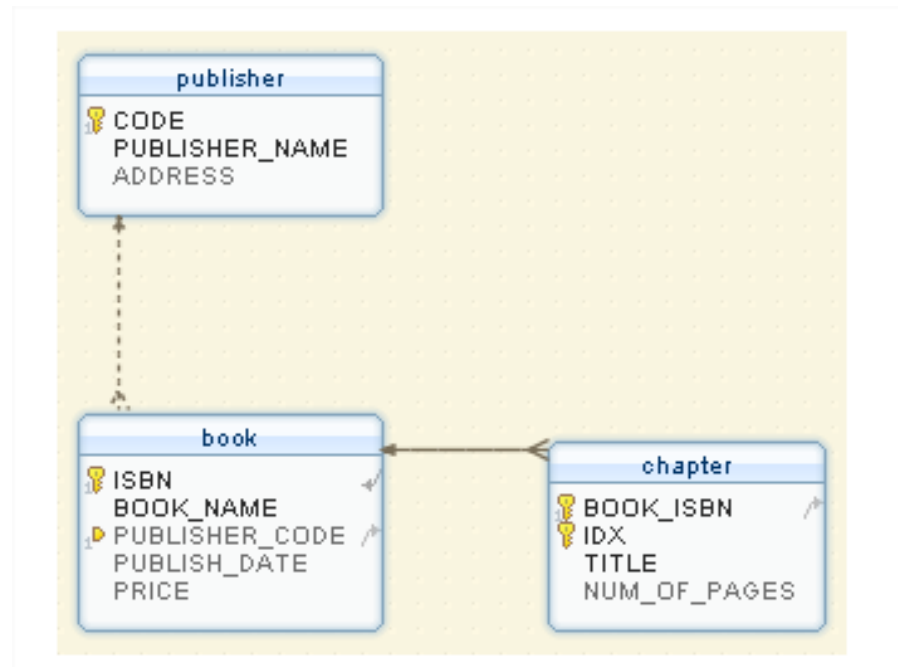
The screenshot shows a Java IDE console window with two tabs: 'Console' and 'Progress'. The 'Console' tab is active, displaying the output of a Java application. The output text is: '<terminated> DatabaseTest [Java Application] C:\Program Files\Java\jdk1.6.0_23\bin\javaw.exe (18/02/2011 00:45:45)' followed by 'Deleted count : 1'.

```
<terminated> DatabaseTest [Java Application] C:\Program Files\Java\jdk1.6.0_23\bin\javaw.exe (18/02/2011 00:45:45)
Deleted count : 1
```

<https://docs.oracle.com/javase/7/docs/api/java/sql/PreparedStatement.html>

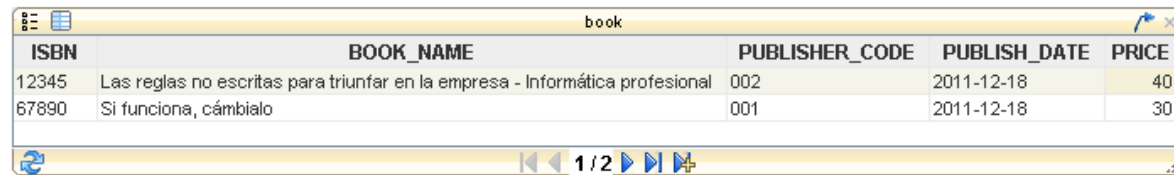
6. ACCESO A BASE DE DATOS: JDBC

■ Ejemplo:



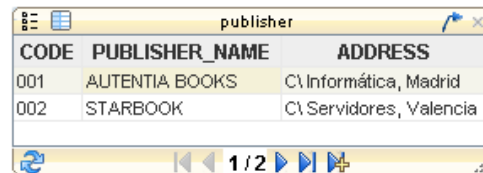
6. ACCESO A BASE DE DATOS: JDBC

Tabla Books



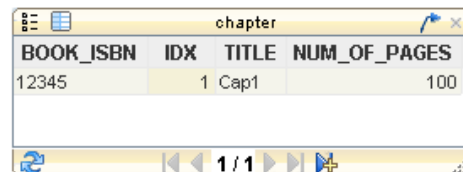
ISBN	BOOK_NAME	PUBLISHER_CODE	PUBLISH_DATE	PRICE
12345	Las reglas no escritas para triunfar en la empresa - Informática profesional	002	2011-12-18	40
67890	Si funciona, cámbialo	001	2011-12-18	30

Tabla Publisher



CODE	PUBLISHER_NAME	ADDRESS
001	AUTENTIA BOOKS	C\Informática, Madrid
002	STARBOOK	C\Servidores, Valencia

Tabla Chapters



BOOK_ISBN	IDX	TITLE	NUM_OF_PAGES
12345	1	Cap1	100

6. ACCESO A BASE DE DATOS: JDBC

```
String isbn = "12345";

PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM BOOK, PUBLISHER WHERE BOOK.PUBLISHER_CODE = PUBLISHER.CODE AND BOOK.ISBN = ?");
stmt.setString(1, isbn);
ResultSet rs = stmt.executeQuery();
Book book = new Book();
if (rs.next()) {
    book.setIsbn(rs.getString("ISBN"));
    book.setName(rs.getString("BOOK_NAME"));
    book.setPublishDate(rs.getDate("PUBLISH_DATE"));
    book.setPrice(rs.getInt("PRICE"));
    Publisher publisher = new Publisher();
    publisher.setCode(rs.getString("PUBLISHER_CODE"));
    publisher.setName(rs.getString("PUBLISHER_NAME"));
    publisher.setAddress(rs.getString("ADDRESS"));
    book.setPublisher(publisher);
}
rs.close();
stmt.close();

List<Chapter> chapters = new ArrayList<Chapter>();
stmt = conn.prepareStatement("SELECT * FROM CHAPTER WHERE BOOK_ISBN = ?");
stmt.setString(1, isbn);
rs = stmt.executeQuery();
while (rs.next()) {
    Chapter chapter = new Chapter();
    chapter.setIndex(rs.getInt("IDX"));
    chapter.setTitle(rs.getString("TITLE"));
    chapter.setNumOfPages(rs.getInt("NUM_OF_PAGES"));
    chapters.add(chapter);
}
book.setChapters(chapters);
rs.close();
stmt.close();
```


6. ACCESO A BASE DE DATOS: JDBC

■ Resultado:

```
BOOK[
  12345
  Las reglas no escritas para triunfar en la empresa
  Informática profesional
  PUBLISHER[002 – STARBOOK – C\ Servidores, Valencia]
  2011-12-18
  40
  CHAPTER[1 – Cap1 – 100]]
```

6. ACCESO A BASE DE DATOS: JDBC

Práctica:

- <https://spring.io/guides/gs/relational-data-access/>
- <https://github.com/spring-petclinic/spring-petclinic-data-jdbc>

7. ACCESO A BASE DE DATOS: HIBERNATE

7. ACCESO A BASE DE DATOS: HIBERNATE

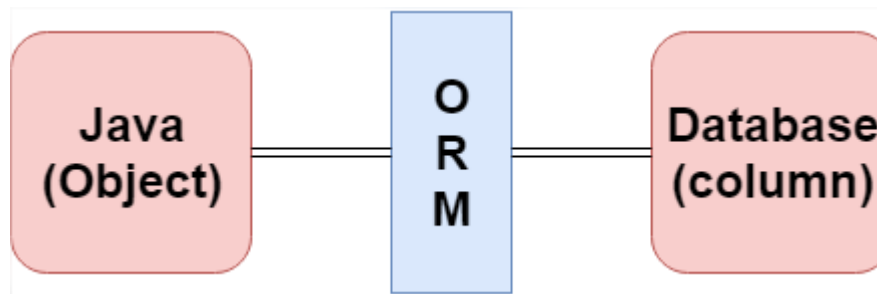
Hibernate es una herramienta de mapeo objeto-relacional (ORM) que facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación, mediante archivos declarativos (XML) o anotaciones en los beans de las entidades que permiten establecer estas relaciones.



7. ACCESO A BASE DE DATOS: HIBERNATE

El patrón de mapeo objeto-relacional (ORM) es una técnica de programación para convertir datos entre el sistema de tipos utilizado en un lenguaje de programación orientado a objetos y la utilización de una base de datos relacional como motor de persistencia.

- En la práctica esto crea una base de datos orientada a objetos virtual, sobre la base de datos relacional
- Posibilita el uso de las características propias de la orientación a objetos (básicamente herencia y polimorfismo)



7. ACCESO A BASE DE DATOS: HIBERNATE

Hibernate utiliza el lenguaje **HQL** (Hibernate Query Language) para la construcción de consultas de forma independiente de la base de datos, utilizando cláusulas:

- FROM
- SELECT
- WHERE
- GROUP BY

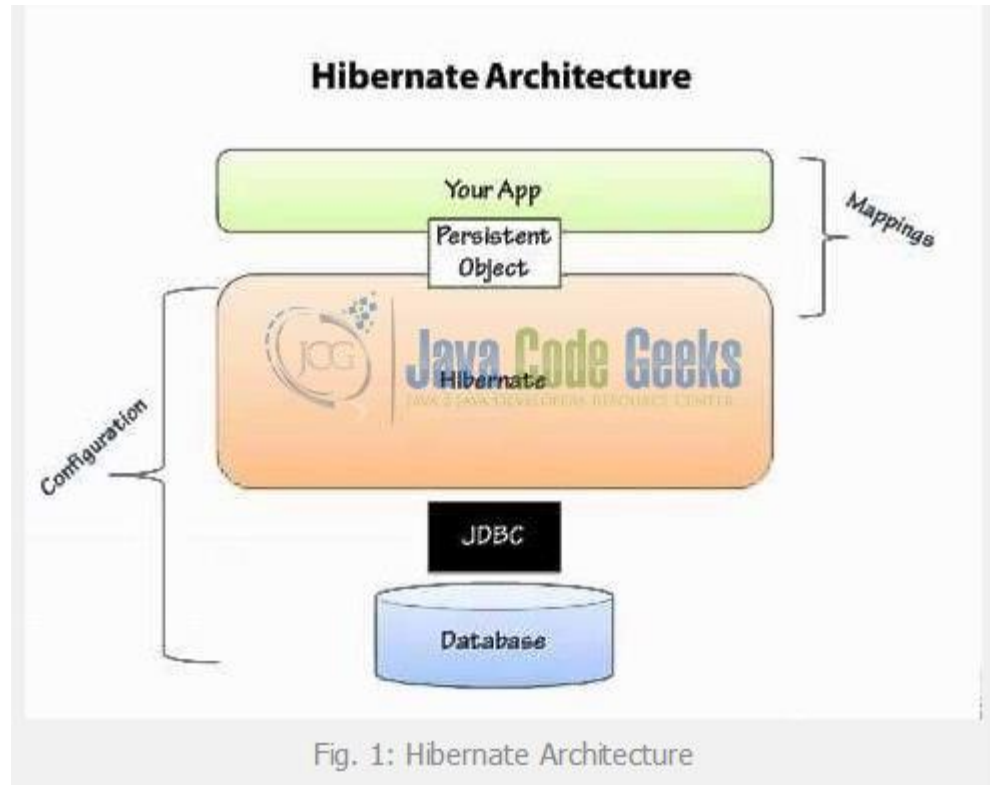
Y funciones:

- Sum()
- Avg()
- Max()
- Min()
- Count()

7. ACCESO A BASE DE DATOS: HIBERNATE

- Provee la facilidad de crear las tablas de BD automáticamente. Con lo que no hay necesidad de crear tablas en la BD manualmente
- Utiliza transacciones para garantizar la consistencia de datos
- Utiliza asociaciones de distintos tipos
 - One-to-One
 - Many-to-One
 - Many-to-Many

7. ACCESO A BASE DE DATOS: HIBERNATE



7. ACCESO A BASE DE DATOS: HIBERNATE

Los elementos de la **arquitectura de Hibernate** son:

- **SessionFactory:** Es una factoría de sesión y cliente del **ConnectionProvider**.
- **Session:** Provee una interfaz entre la aplicación y los datos guardados en la BD. Es un objeto de corta vida que se enlaza con la conexión JDBC
- **Transaction:** Especifica la unidad atómica de trabajo y es un parámetro opcional en el framework Hibernate
- **ConnectionProvider:** Es una factoría de las conexiones JDBC y abstrae la aplicación del **DriverManager** o del **DataSource**.
- **TransactionFactory:** Es una factoría de objetos **Transaction**

7. ACCESO A BASE DE DATOS: HIBERNATE

Las principales **anotaciones** de Hibernate son:

- **@Entity**: Marca una clase como Entidad Hibernate (Mapeado de Clase)
- **@Table (name="valor")**: Mapea esta clase con una tabla de BD con nombre "valor". Si el nombre no coincide, se mapeará con la tabla que tenga el mismo nombre que la clase Java.
- **@Id**: Marca este campo de clase como columna primary key
- **@GeneratedValue**: Indica a la BD que genere un valor para este campo automáticamente
- **@Column (name="valor")**: Mapea este campo como columna de tabla
- **@ManyToMany**: Marca este campo como relación de N:M
- **@JoinTable (name, joinColumns, inverseJoinColumns)**: Para realizar esta relación de N:M, se necesitarán los campos relacionados
- **@JoinColumn (name)**: Mapea una columna de una Join con la @JoinTable de la relación anterior

7. ACCESO A BASE DE DATOS: HIBERNATE

Prácticas:

- Herramientas:

<https://dev.mysql.com/get/Downloads/MySQLInstaller/mysql-installer-community-5.7.27.0.msi>

- Aplicación de Estudiantes:

<https://examples.javacodegeeks.com/enterprise-java/hibernate/hibernate-example-code-using-annotations/>

8. ACCESO A BASE DE DATOS: JPA

8. ACCESO A BASE DE DATOS: JPA

Java Persistence API (JPA) es la API de persistencia desarrollada para Aplicaciones Web Java.

Es un framework de Java que maneja datos relacionales en aplicaciones.

Persistencia en este contexto cubre 3 áreas:

- La API en si misma, definida en el paquete `java.persistence`
- El lenguaje de consulta Java Persistence Query Language (JPQL)
- Metadatos objeto/relacional



8. ACCESO A BASE DE DATOS: JPA

El **objetivo** del diseño esta API es:

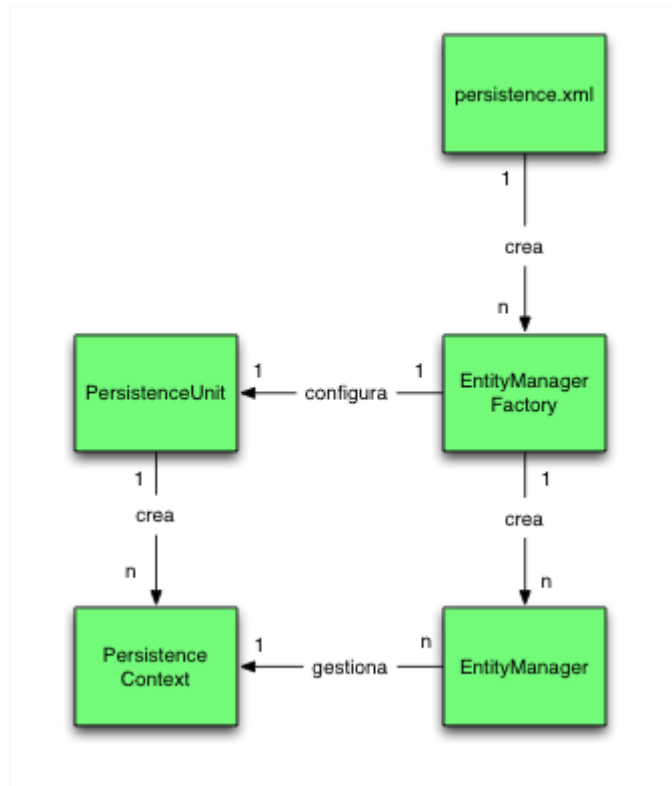
- No perder las **ventajas de la orientación a objetos** al interactuar con una base de datos (siguiendo el patrón de mapeo objeto-relacional)
- Permitir usar objetos regulares (conocidos como **POJO**)

8. ACCESO A BASE DE DATOS: JPA

- Una **entidad de persistencia (Entity)** es una clase Java ligera, cuyo estado es persistido de manera asociada a una tabla en una base de datos relacional
- Las instancias de estas entidades corresponden a **un registro** (conjunto de datos representados en una fila) **en la tabla**
- Normalmente las **entidades están relacionadas con otras entidades**, y estas relaciones son expresadas a través de **metadatos objeto/relacional**.
- Los metadatos del objeto/relacional pueden ser especificados directamente en el fichero de la clase, usando **anotaciones** o en un documento descriptivo **XML**, el cual es distribuido junto con la aplicación

8. ACCESO A BASE DE DATOS: JPA

■ Estructura JPA



8. ACCESO A BASE DE DATOS: JPA

```
1. <persistence xmlns="http://java.sun.com/xml/ns/persistence"
2.   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3.   xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
4.     http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd"
5.   version="2.0">
6.   <persistence-unit name="UnidadPersonas">
7.     <class>es.curso.bo.Persona</class>
8.     <properties>
9.       <property name="hibernate.show_sql" value="true" />
10.      <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect" />
11.      <property name="javax.persistence.jdbc.driver" value="com.mysql.jdbc.Driver" />
12.      <property name="javax.persistence.jdbc.user" value="root" />
13.      <property name="javax.persistence.jdbc.password" value="jboss" />
14.      <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost/jpa" />
15.
16.     </properties>
17.
18.   </persistence-unit>
19. </persistence>
```

8. ACCESO A BASE DE DATOS: JPA

```
1. package com.arquitecturajava;
2.
3. import javax.persistence.EntityManager;
4. import javax.persistence.EntityManagerFactory;
5. import javax.persistence.Persistence;
6.
7. import es.curso.bo.Persona;
8.
9. public class Principal01Add {
10.
11.     public static void main(String[] args) {
12.
13.         Persona yo = new Persona("pedro",25);
14.         EntityManagerFactory emf =
15.             Persistence.createEntityManagerFactory("UnidadPersonas");
16.         EntityManager em = emf.createEntityManager();
17.         try {
18.             em.getTransaction().begin();
19.             em.persist(yo);
20.             em.getTransaction().commit();
21.         } catch (Exception e) {
22.
23.             e.printStackTrace();
24.         } finally {
25.             em.close();
26.
27.         }
28.     }
29. }
```

8. ACCESO A BASE DE DATOS: JPA

```
1. package es.curso.bo;
2.
3. import javax.persistence.Entity;
4. import javax.persistence.Id;
5.
6. @Entity
7. public class Persona {
8.
9.     @Id
10.    private String nombre;
11.    private int edad;
12.    public Persona() {
13.        super();
14.    }
15.
16.    public Persona(String nombre, int edad) {
17.        super();
18.        setEdad(edad);
19.        setNombre(nombre);
20.    }
21.
22.    public String getNombre() {
23.
24.        return nombre;
25.    }
26.    public void setNombre(String nombre) {
27.        this.nombre = nombre;
28.    }
29.
30.
31.    public int getEdad() {
32.
33.        return edad;
34.    }
35.    public void setEdad(int edad) {
36.        this.edad = edad;
37.    }
38. }
```

8. ACCESO A BASE DE DATOS: JPA

```
40.  @Override
41.  public int hashCode() {
42.      //simplificacion
43.      return nombre.hashCode();
44.  }
45.
46.  @Override
47.  public boolean equals(Object obj) {
48.      //simplificacion
49.      Persona nueva= (Persona)obj;
50.      return nueva.getNombre().equals(this.getNombre());
51.  }
52.  }
53.  }
```

8. ACCESO A BASE DE DATOS: JPA

Los métodos de EntityManager son:

- Persist(): es el encargado de almacenar nuevas entidades en BD

```
1.  public static void main(String[] args) {
2.
3.      Persona yo = new Persona("pedro", 25);
4.      EntityManagerFactory emf =
5.      Persistence.createEntityManagerFactory("UnidadPersonas");
6.      EntityManager em = emf.createEntityManager();
7.      try {
8.          em.getTransaction().begin();
9.          em.persist(yo);
10.         em.getTransaction().commit();
11.     } catch (Exception e) {
12.
13.         e.printStackTrace();
14.     } finally {
15.         em.close();
16.
17.     }
18.
19. }
```

8. ACCESO A BASE DE DATOS: JPA

- Contains(): comprueba si una entidad esta gestionada por el EntityManager devolviendo true o false según corresponda

```
1. public static void main(String[] args) {  
2.  
3.     Persona yo = new Persona("pedro",25);  
4.     EntityManagerFactory emf =  
5.     Persistence.createEntityManagerFactory("UnidadPersonas");  
6.     EntityManager em = emf.createEntityManager();  
7.     try {  
8.         em.getTransaction().begin();  
9.  
10.        System.out.println(em.contains(yo));  
11.  
12.        em.persist(yo);  
13.  
14.        System.out.println(em.contains(yo));  
15.  
16.        em.getTransaction().commit();  
17.    } catch (Exception e) {  
18.  
19.        e.printStackTrace();  
20.    } finally {  
21.        em.close();  
22.  
23.    }  
24.  
25. }
```

8. ACCESO A BASE DE DATOS: JPA

- Find(): se encarga de localizar una Entidad a través de su clave primaria y su tipo de Entidad (*hay que sobrecargar correctamente los métodos equals y hascode)

```
1. public static void main(String[] args) {  
2.  
3.     EntityManagerFactory emf = Persistence  
4.         .createEntityManagerFactory("UnidadPersonas");  
5.     EntityManager em = emf.createEntityManager();  
6.     Persona yo = em.find(Persona.class, "cecilio");  
7.     System.out.println(yo.getNombre());  
8.     System.out.println(yo.getEdad());  
9.  
10. }
```

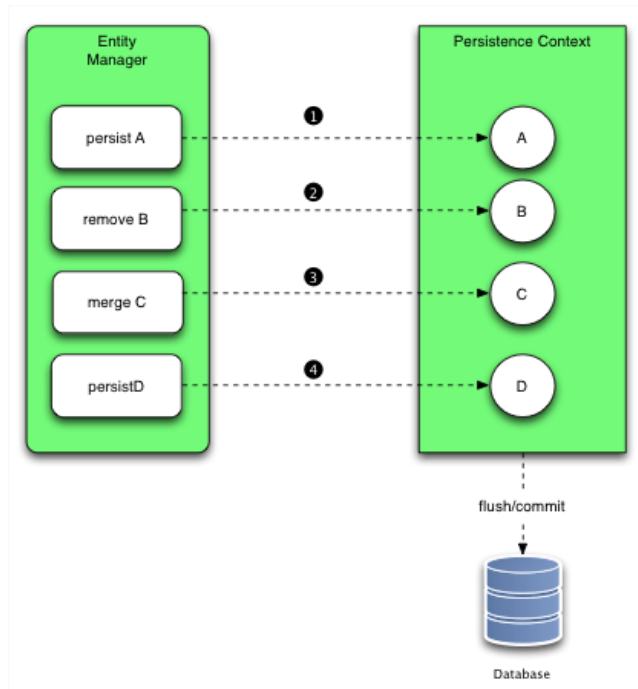
8. ACCESO A BASE DE DATOS: JPA

- Remove(): elimina una entidad de la BD

```
1.  public static void main(String[] args) {  
2.  
3.      EntityManagerFactory emf = Persistence  
4.      .createEntityManagerFactory("UnidadPersonas");  
5.      EntityManager em = emf.createEntityManager();  
6.      try {  
7.  
8.          Persona yo = em.find(Persona.class, "cecilio");  
9.          em.getTransaction().begin();  
10.         em.remove(yo);  
11.         em.getTransaction().commit();  
12.     } catch (Exception e) {  
13.         em.close();  
14.     }  
15.  
16. }
```


8. ACCESO A BASE DE DATOS: JPA

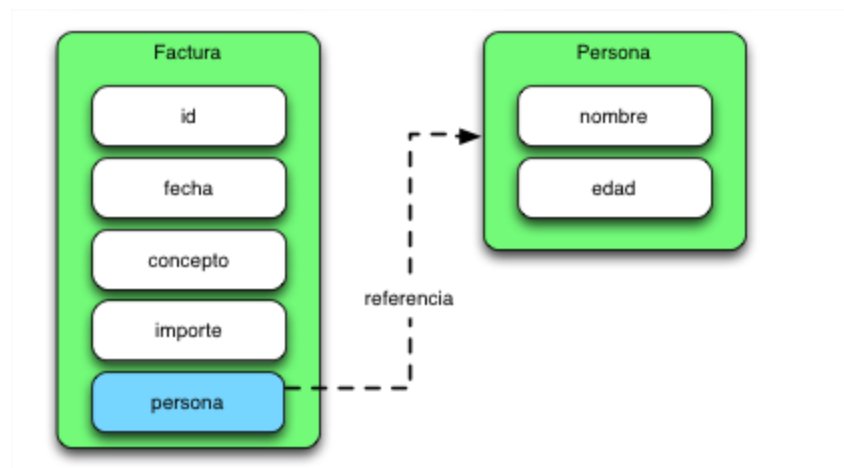
- Flush(): Es el encargado de sincronizar el PersistenceContext contra la BD, ya que el EntityManager irá almacenando las modificaciones que nosotros realizamos sobre las entidades, para mas tarde persistirlas contra la BD con un flush o con el commit de una transacción



8. ACCESO A BASE DE DATOS: JPA

Los tipos de relación de JPA son:

- **@ManyToOne**: Es la relación entre tablas de 1 a n. Una persona tiene n facturas y una factura pertenece a una persona. Aquí vamos a definir la relación entre Factura y Persona.



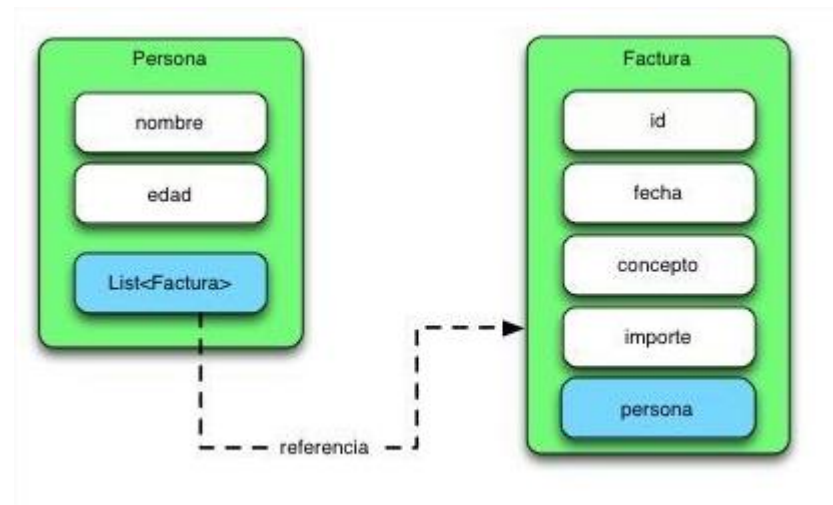
8. ACCESO A BASE DE DATOS: JPA

```
1. package es.curso.bo;
2.
3. import java.util.Date;
4.
5. import javax.persistence.Entity;
6. import javax.persistence.Id;
7. import javax.persistence.JoinColumn;
8. import javax.persistence.ManyToOne;
9.
10. @Entity
11. public class Factura {
12.
13.     @Id
14.     private int id;
15.     private Date fecha;
16.     private double importe;
17.     private String concepto;
18.
19.     @ManyToOne
20.     @JoinColumn(name="persona_nombre")
21.     private Persona persona;
22.
23.     public Factura() {
24.         super();
25.         // TODO Auto-generated constructor stub
26.     }
27.
28.     public Factura(int id, Date fecha, double importe, String concepto, Persona persona) {
29.         super();
30.         this.id=id;
31.         this.fecha = fecha;
32.         this.importe = importe;
33.         this.concepto = concepto;
34.         this.persona=persona;
35.     }
36.
37.     public Persona getPersona() {
38.         return persona;
39.     }
40.
41.     public void setPersona(Persona persona) {
42.         this.persona = persona;
43.     }
44. }
```

8. ACCESO A BASE DE DATOS: JPA

Los tipos de relación de JPA son:

- @OneToMany: Aquí vamos a definir la relación entre Persona y Factura

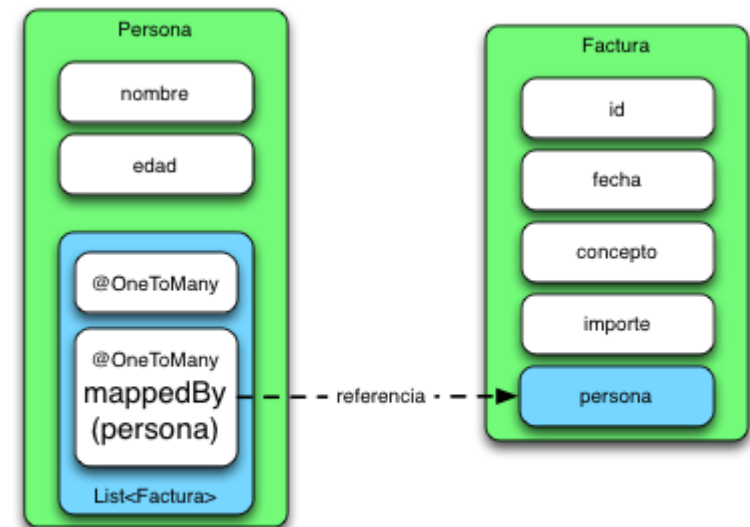


8. ACCESO A BASE DE DATOS: JPA

```
1.  @Entity
2.  public class Persona {
3.
4.      @Id
5.
6.      private String nombre;
7.
8.      private int edad;
9.
10.     @OneToMany(mappedBy="persona" )
11.     private List<Factura> listaFacturas;
12.
13.     public Persona() {
14.         super();
15.         listaFacturas=new ArrayList<Factura>();
16.     }
17.
18.     public List<Factura> getListaFacturas() {
19.         return listaFacturas;
20.     }
21.
22.     public void addFactura(Factura f) {
23.
24.         listaFacturas.add(f);
25.     }
26.
27.     public void setListaFacturas(List<Factura> listaFacturas) {
28.         this.listaFacturas = listaFacturas;
29.     }
30.
31.     public Persona(String nombre, int edad) {
32.         super();
33.         this.nombre = nombre;
34.         this.edad = edad;
35.         listaFacturas=new ArrayList();
36.     }
```

8. ACCESO A BASE DE DATOS: JPA

```
1. @Entity
2. public class Persona {
3.
4.     @Id
5.
6.     private String nombre;
7.
8.     private int edad;
9.
10.    @OneToMany(mappedBy="persona" )
11.    private List<Factura> listaFacturas;
12.
13.    public Persona() {
14.        super();
15.        listaFacturas=new ArrayList<Factura>();
16.    }
17.
18.    public List<Factura> getListaFacturas() {
19.        return listaFacturas;
20.    }
21.
22.    public void addFactura(Factura f) {
23.
24.        listaFacturas.add(f);
25.    }
26.
27.    public void setListaFacturas(List<Factura> listaFacturas) {
28.        this.listaFacturas = listaFacturas;
29.    }
30.
31.    public Persona(String nombre, int edad) {
32.        super();
33.        this.nombre = nombre;
34.        this.edad = edad;
35.        listaFacturas=new ArrayList();
36.    }
37.    public String getNombre() {
38.        return nombre;
39.    }
40.    public void setNombre(String nombre) {
41.        this.nombre = nombre;
42.    }
43.
44.    public int getEdad() {
45.        return edad;
46.    }
47.    public void setEdad(int edad) {
48.        this.edad = edad;
49.    }
```



8. ACCESO A BASE DE DATOS: JPA

Prácticas:

- Aplicación de Vendedores con JPA:

<https://spring.io/guides/gs/accessing-data-jpa/>

- Aplicación de trabajadores con Spring Data JPA:

<https://examples.javacodegeeks.com/enterprise-java/spring/spring-data-jpa-example/>

- Aplicación de Recetas con JPA:

<https://github.com/springframeworkguru/spring5-recipe-app/tree/display-image-from-db>

9. SERVICIOS REST

9. SERVICIOS REST

- Un servicio REST (Representational State Transfre) no es una arquitectura de software sino un conjunto de restricciones con las que podemos crear un estilo de arquitectura software, la cual podremos usar para crear aplicaciones web respetando HTTP.
- Es un estándar lógico y eficiente para la creación de servicios web



9. SERVICIOS REST

Las **restricciones** que definen a un sistema **RESTful** serían:

- **Cliente-servidor:** esta restricción mantiene al cliente y al servidor débilmente acoplados. Esto quiere decir que el cliente no necesita conocer los detalles de implementación del servidor y el servidor se “despreocupa” de como son usados los datos que envía al cliente
- **Sin estado:** aquí decimos que cada petición que recibe el servidor debería ser independiente, es decir, no es necesario mantener sesiones
- **Cacheable:** debe admitir un sistema de almacenamiento en caché. La infraestructura de red debe soportar una caché de varios niveles. Este almacenamiento evitará repetir varias conexiones entre el servidor y el cliente para recuperar un mismo recurso

9. SERVICIOS REST

- **Interfaz uniforme:** define una interfaz genérica para administrar cada interacción que se produzca entre el cliente y el servidor de manera uniforme, lo cual simplifica y separa la arquitectura. Esta restricción indica que **cada recurso** del servicio REST debe tener una **única dirección URI**
- **Sistema de capas:** el servidor puede disponer de **varias capas** para su implementación. Esto ayuda a mejorar la escalabilidad, el rendimiento y la seguridad

9. SERVICIOS REST

Las operaciones HTTP más importantes que nos permitirán manipular los recursos son cuatro:

- GET: para consultar y leer
- POST: para crear
- PUT: para editar
- DELETE: para eliminar

9. SERVICIOS REST

- El uso de **hipermedios** (término que en el ámbito de las páginas web define el conjunto de procedimientos para crear contenidos que contengan texto, imagen, vídeo, audio y otros métodos de información) para permitir al usuario **navegar** por los **distintos recursos** de una API REST a través de enlaces HTML.
- Lo más importante de nuestro servido o API REST no es el lenguaje en el que se implemente (**independiente del lenguaje**) sino que las **respuestas** a las peticiones se hagan en **XML o JSON**, ya que es el lenguaje de intercambio de información más usado
- Algunos frameworks para implementar API REST son: JAX-RS y Spring Boot (Java), Django REST (Python), Laravel (PHP) o Restify (Node.js)

9. SERVICIOS REST

- El uso de **hipermedios** (término que en el ámbito de las páginas web define el conjunto de procedimientos para crear contenidos que contengan texto, imagen, vídeo, audio y otros métodos de información) para permitir al usuario **navegar** por los **distintos recursos** de una API REST a través de enlaces HTML.
- Lo más importante de nuestro servido o API REST no es el lenguaje en el que se implemente (**independiente del lenguaje**) sino que las **respuestas** a las peticiones se hagan en **XML o JSON**, ya que es el lenguaje de intercambio de información más usado
- Algunos frameworks para implementar API REST son: JAX-RS y Spring Boot (Java), Django REST (Python), Laravel (PHP) o Restify (Node.js)

9. SERVICIOS REST

- Creación de un API REST:

<https://www.oscarblancarteblog.com/2018/06/25/creando-un-api-rest-en-java-parte-1/>

- Creación de métodos HTTP

<https://www.oscarblancarteblog.com/2018/12/03/metodos-http-rest/>

9. SERVICIOS REST

- Ejemplo Spring + REST + JPA:

<https://spring.io/guides/gs/accessing-data-rest/>

- Practica REST Cliente-Vendedor (PostMan):

<https://github.com/springframeworkguru/spring5-mvc-rest/tree/vendor-api>

- Práctica REST Veterinario:

<https://github.com/spring-petclinic/spring-petclinic-rest>

10. STRUTS FRAMEWORK

10. STRUTS FRAMEWORK

Struts Framework (Apache Struts), es una herramienta de soporte para el desarrollo de aplicaciones Web del patrón MVC en Java.

Struts permite reducir el tiempo de desarrollo

Su carácter de software libre y su compatibilidad con todas las plataformas de JEE lo convierten en una herramienta altamente disponible



10. STRUTS FRAMEWORK

Struts implementa de la siguiente manera los componentes del patron **Modelo-Vista-Controlador**:

- El **Controlador** es un servlet (Action), de una clase proporcionada por Struts. Será necesario configurar la aplicación web (a través del fichero web.xml) para que todas las peticiones del usuario se redirijan a este servlet
 - Este despacha las peticiones del usuario a la clase adecuada para ejecutar la acción.

10. STRUTS FRAMEWORK

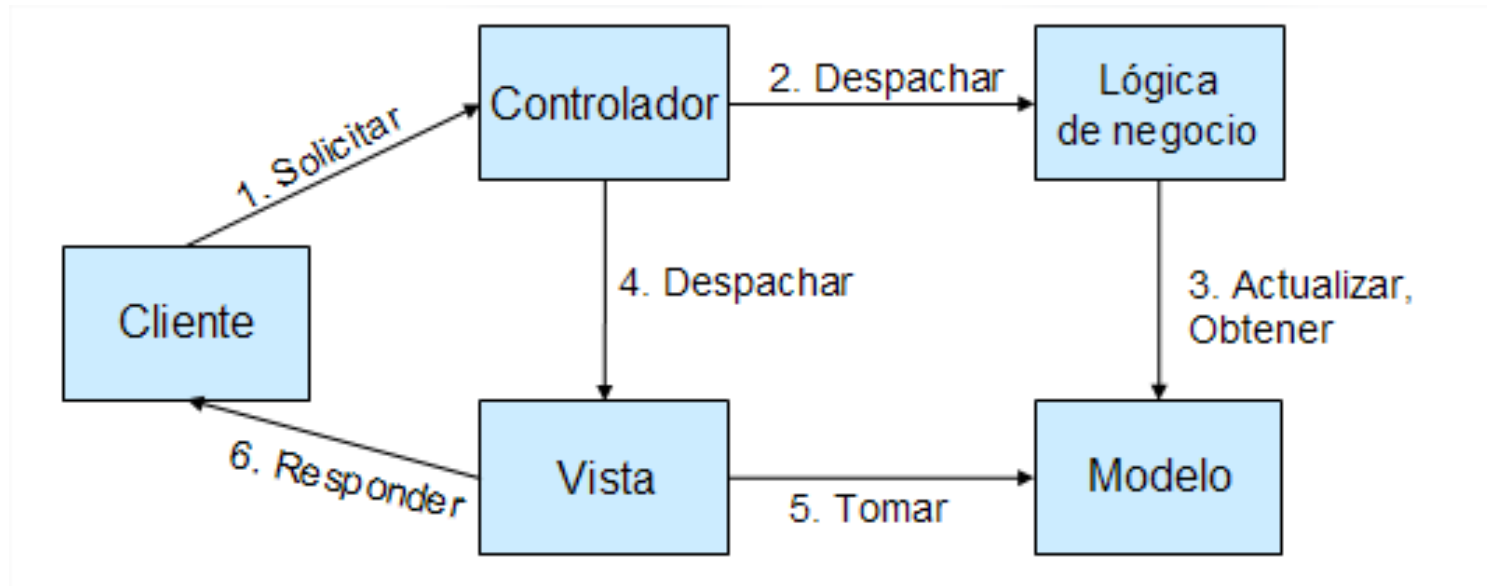
- La **Vista** se implementará normalmente mediante páginas JSP. Se ofrecen 2 herramientas para ayudar a la presentación de datos:
 - ActionForm: son clases que capturan los datos introducidos en formularios y permiten su validación
 - Librería de etiquetas: permiten mostrar errores y facilitar el trabajo con formularios
- La implementación del **modelo** corre a cargo del desarrollador, ya que es propio de la capa de negocio y no está dentro del ámbito de Struts

10. STRUTS FRAMEWORK

El ciclo de vida que se sigue cuando Struts procesa una petición HTTP es:

1. El cliente realiza la **petición**, que recibe el controlador de Struts (**Action**)
2. El controlador **despacha la petición**, identificando la acción y disparando la lógica de negocio apropiada (servicio)
3. La lógica de negocio **actualiza el modelo** y obtiene datos del mismo almacenándolos en **beans**
4. En función del valor devuelto, el controlador elige la **siguiente vista** a mostrar
5. **La vista toma los datos** obtenidos por la lógica de negocio
6. **La vista muestra los datos** en el cliente

10. STRUTS FRAMEWORK



10. STRUTS FRAMEWORK

- **Web.xml:** En este fichero de configuración web tendremos que definir los servlets que utilizaremos y sus redirecciones

```
<!-- Definir el servlet que hace de controlador -->
<servlet>
  <servlet-name>controlador</servlet-name>
  <servlet-class>org.apache.struts.action.ActionServlet</servlet-class>
  <!-- Podemos cambiar la carpeta por defecto del fich. de configuracion
       (aunque no es algo que se suela hacer a menudo) -->
  <init-param>
    <param-name>config</param-name>
    <param-value>/WEB-INF/config/struts-config.xml</param-value>
  </init-param>
  <load-on-startup>2</load-on-startup>
</servlet>
<!-- redirigir ciertas peticiones al controlador -->
<servlet-mapping>
  <servlet-name>controlador</servlet-name>
  <url-pattern>*.do</url-pattern>
</servlet-mapping>
```

10. STRUTS FRAMEWORK

- **Acciones (ActionServlet):** son las clases encargadas de ejecutar las acciones

```
package acciones;
import javax.servlet.http.*;
import org.apache.struts.action.*;
public class AccionLogin extends Action
{
    public ActionForward execute(ActionMapping mapping, ActionForm form,
                                HttpServletRequest request, HttpServletResponse response)
                                throws Exception {
        boolean usuarioOK;
        //obtener login y password y autenticar al usuario
        //si es correcto, poner usuarioOK a 'true'
        ...
        //dirigirnos a la vista adecuada según el resultado
        if (usuarioOK)
            return mapping.findForward("OK");
        else
            return mapping.findForward("errorUsuario");
    }
}
```


10. STRUTS FRAMEWORK

- **struts-config.xml**: es el fichero de configuración de Struts que indica el flujo de la navegación de las acciones (mapeado de peticiones y acciones)

```
<?xml version = "1.0" encoding = "ISO-8859-1"?>
<!DOCTYPE struts-config PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 1.1//EN"
    "http://jakarta.apache.org/struts/dtds/struts-config_1_1.dtd">
<struts-config>
<!--definición de otros elementos del fichero de configuración -->
    ...
    ...
<!--definición de acciones -->
<action-mappings>
    <!-- hacer login -->
    <action path="/login" type="acciones.AccionLogin">
        <forward name="OK" path="/personal.jsp"/>
        <forward name="errorUsuario" path="/error.html"/>
    </action>
    <!-- definición de otras acciones -->
    ...
</action-mappings>
</struts-config>
```

10. STRUTS FRAMEWORK

- **Tratamiento de excepciones:** desde el struts-config.xml también podemos definir que cuando salten determinadas excepciones vayan a determinadas vistas para mostrarlas

```
<?xml version = "1.0" encoding = "ISO-8859-1"?>
<!DOCTYPE struts-config PUBLIC "-//Apache Software Foundation//DTD Struts
Configuration 1.1//EN" "http://jakarta.apache.org/struts/dtds/struts-config_1_1.dtd">
<struts-config>

    <global-exceptions>
        <exception type="es.ua.jtech.ejemplos.ExcepcionEjemplo"
            key="error.ejemplo"
            path="/error.jsp"/>
    </global-exceptions>

    <global-forwards>
        <forward name="errorUsuario" path="/error.html"/>
    </global-forwards>
    ...
</struts-config>
```

10. STRUTS FRAMEWORK

- **Tiles-definitions.xml:** es un plugin de Struts y se utiliza para definir vistas compuestas de un layout

```
<!-- ##### -->
<!-- ## PAGINES PER A LA GESTIO D'USUARIS ## -->
<!-- ##### -->

<!-- pagina de seleccio d'entitat per a usuari -->
<definition name="pages.seleccioEntitatUsuari" extends="site.mLayout" >
  <put name="head" value="/WEB-INF/jsp/includes/header.jsp" />
  <put name="subhead" value="/WEB-INF/jsp/includes/subhead.jsp" />
  <put name="ariadna" value="/WEB-INF/jsp/includes/blank.jsp"/>
  <put name="menuLeft" value="/WEB-INF/jsp/includes/menuDisabled.jsp" />
  <put name="body" value="/WEB-INF/jsp/formularis/usuaris/seleccioEntitat.jsp" />
  <put name="footer" value="/WEB-INF/jsp/includes/footer.jsp" />
</definition>
```

10. STRUTS FRAMEWORK

- **ActionForm:** sirve para obtener los datos introducidos en los formularios de una forma sencilla. Se definen en el struts-config.xml y hay que implementarlos en Java

```
<form-beans>
  <form-bean name="FormLogin" type="acciones.forms.FormLogin">
</form-beans>
```

```
<action path="/login" type="acciones.AccionLogin"
        name="FormLogin" scope="session"
        validate="true" input="/index.jsp">
  <forward name="OK" path="/personal.jsp"/>
  <forward name="errorUsuario" path="/error.html"/>
</action>
```

10. STRUTS FRAMEWORK

```
package acciones.formularios;
import org.apache.struts.action.*;
import javax.servlet.http.HttpServletRequest;

public class FormLogin extends ActionForm {
    private String login;
    private String password;

    public void setLogin(String login) {
        this.login = login;
    }
    public String getLogin() {
        return login;
    }

    public ActionErrors validate(ActionMapping mapping,
                                HttpServletRequest request) {
        ActionErrors errores = new ActionErrors();
        if ((getLogin()==null) || (getLogin().equals(""))){
            errores.add(ActionMessages.GLOBAL_MESSAGE
                        , new ActionMessage("error.requerido.usuario"));
        }
        return errores;
    }
}
```

10. STRUTS FRAMEWORK

- **Taglibs de Struts:** desde la vista se pueden utilizar estas etiquetas para aprovechar la potencia de Struts, definir acciones al controlador y poder utilizar el modelo desde las JSPs

```
<%@ taglib uri="http://struts.apache.org/tags-html" prefix="html" %>
<html>
<head><title>Ejemplo de tag form</title></head>
<body>
  <html:form action="/registrar">
    <!-- aquí vienen los campos. Ahora veremos cómo se definen -->
  </html:form>
</body>
</html>
```

```
<html:form action="/registrar">
  <html:submit>Registrar nuevo usuario</html:submit>
  <html:cancel>Cancelar</html:cancel>
</html:form>
```

10. STRUTS FRAMEWORK

- Los **campos de texto** de las etiquetas tendrán que estar definidos en el ActionForm con su correspondiente propiedad

```
<html:form action="/registrar">
  <html:text property="login">Introduce aquí el nombre que deseas tener</html:text>
  <html:password property="password"/>
  <html:submit>Registrar nuevo usuario</html:submit>
  <html:cancel>Cancelar</html:cancel>
</html:form>
```

```
<html:textarea property="descripcion" cols="40" rows="6"/>
```

```
<html:radio property="sexo" value="hombre"/> hombre
<html:radio property="sexo" value="mujer"/> mujer
```

10. STRUTS FRAMEWORK

- Las **listas o combos**, se les pasará desde el Action como atributo de la petición `HttpServletRequest` para que aparezcan rellenos o desde el `ActionForm`

```
String[] lista = { "Hombre", "Mujer", "No especificado"}  
request.setAttribute("listaSexos", lista);
```

```
public class FormRegistro {  
    ...  
    private static String[] listaSexos = { "Hombre", "Mujer", "No especificado"};  
  
    public String[] getListasSexos() {  
        return listaSexos;  
    }  
}
```

```
<html:select property="sexo">  
    <html:options value="listaSexos"/>  
</html:select>
```

```
...  
<html:select property="sexo">  
    <html:options property="listaSexos"/>  
</html:select>  
...
```


10. STRUTS FRAMEWORK

- Para acciones donde haya que crear etiquetas iguales con distintos value desde el ActionForm se utiliza JSTL (**JavaServer Pages Standard Tag Library**), ya que es más potente y permite hacer esto que Struts no permite

```
<%@ taglib uri="http://struts.apache.org/tags-html-el" prefix="html-el" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
...
<c:forEach items="${FormRegistro.listaSexos}" var="sexo">
  <html-el:radio property="sexo" value="${sexo}"/> ${sexo}
</c:forEach>
```

```
<c:forEach items="${FormRegistro.listaAficiones}" var="aficion">
  <html-el:multibox property="aficiones">${aficion}</html-el:multibox> ${aficion}
</c:forEach>
```

Más información JSTL:

http://www.ciberaula.com/articulo/introduccion_jstl

10. STRUTS FRAMEWORK

- **Validation.xml:** se utilizan para validar los formularios del ActionForm. Hay que añadirlo al struts-config.xml y la librería como dependencia

```
<plug-in className="org.apache.struts.validator.ValidatorPlugIn">
  <set-property property="pathnames"
    value="/WEB-INF/validator-rules.xml,/WEB-INF/validation.xml"/>
</plug-in>
```

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!DOCTYPE form-validation PUBLIC
  "-//Apache Software Foundation//DTD Commons Validator
  Rules Configuration 1.0//EN"
  "http://jakarta.apache.org/commons/dtds/validator_1_0.dtd">
<form-validation>
  <formset>
    <form name="registro">
      <field property="nombre" depends="required,minlength">
        <var>
          <var-name>minlength</var-name>
          <var-value>6</var-value>
        </var>
      </field>
      ...
    </form>
  </formset>
  ...
</form-validation>
```

10. STRUTS FRAMEWORK

Validador	Significado	Parámetros
required	dato requerido y no vacío (no todo blancos)	ninguno
mask	emparejar con una expresión regular	mask: e.r. a cumplir. Se usan las librerías de Jakarta RegExp.
intRange, longRange, floatRange, doubleRange	valor dentro de un rango	min, max. Estos validadores dependen respectivamente de int,long,... por tanto en el depends de por ejemplo un intRange debe aparecer también int.
maxLength	máxima longitud para un dato	maxLength
minLength	longitud mínima para un dato	minLength
byte,short, integer, long, double, float	dato válido si se puede convertir al tipo especificado	ninguno
date	verificar fecha	datePattern, formato de fecha especificado como lo hace java.text.SimpleDateFormat. Si se utiliza el parámetro datePatternStrict se verifica el formato de modo estricto. Por ejemplo, si el formato especifica dos días para el mes, hay que poner 08, no valdría con 8, que sí valdría con datePattern
creditCard	verificar número de tarjeta de crédito	ninguno
email	verificar dirección de e-mail	ninguno
url	verificar URL	Varios. Consultar documentación de Struts

10. STRUTS FRAMEWORK

- En la propia validación, se puede poner el mensaje de error que queremos que aparezca cuando no se cumpla la validación:

```
<form name="registro">
  <field property="nombre" depends="required,minlength">
    <msg name="required" key="campo.nombre.noexiste"/>
    <var>
      <var-name>minlength</var-name>
      <var-value>6</var-value>
    </var>
  </field>
  ...
</form>
```

Y en el fichero `.properties`, tendríamos:

```
campo.nombre.noexiste=el nombre no puede estar vacío
```

```
public class LoginForm extends org.apache.struts.validator.action.ValidatorForm {
```

10. STRUTS FRAMEWORK

- Ejercicio:

<https://examples.javacodegeeks.com/enterprise-java/struts-2/struts-jsp-example/>

GRACIAS



BIBLIOGRAFÍA

- <https://openwebinars.net/blog/que-es-spring-framework/>
- <https://www.javatutoriales.com/2010/09/spring-parte-1-introduccion.html>
- <https://spring.io/guides>
- <https://picandocodigo.net/2010/introduccion-a-spring-framework-java/>
- https://es.wikipedia.org/wiki/Inversi%C3%B3n_de_control
- https://es.wikipedia.org/wiki/Inyecci%C3%B3n_de_dependencias
- <http://www.jtech.ua.es/j2ee/publico/spring-2012-13/sesion01-apuntes.html>
- <https://es.wikipedia.org/wiki/Maven>
- <https://www.arquitecturajava.com/que-es-spring-boot/>
- <https://spring.io/projects/spring-boot>
- <https://www.oscarblancarteblog.com/2018/07/17/spring-boot-relacion-los-microservicios/>
- <https://docs.spring.io/spring/docs/5.1.8.RELEASE/spring-framework-reference/web.html#mvc>
- <https://es.wikipedia.org/wiki/Modelo%E2%80%93vista%E2%80%93controlador>
- <https://www.ecodeup.com/patrones-de-diseno-en-java-mvc-dao-y-dto/>
- <https://ingeniods.wordpress.com/2013/05/09/interface-patron-de-diseno/>
- https://es.wikipedia.org/wiki/Java_Database_Connectivity
- <https://www.adictosaltrabajo.com/2011/02/25/tutorial-basico-jdbc/>
- <https://es.wikipedia.org/wiki/Hibernate>
- <https://blog.educacionit.com/2013/02/07/que-es-java-hibernate/>
- <http://www.davidmarco.es/hibernate>

BIBLIOGRAFÍA (II)

- https://es.wikipedia.org/wiki/Java_Persistence_API
- <https://www.arquitecturajava.com/ejemplo-de-jpa/>
- <https://www.arquitecturajava.com/jpa-entitymanager-metodos/>
- <https://www.arquitecturajava.com/jpa-manytoone/>
- <https://www.arquitecturajava.com/jpa-onetomany/>
- https://es.wikipedia.org/wiki/Mapeo_objeto-relacional
- <https://www.idento.es/blog/desarrollo-web/que-es-una-api-rest/>
- <https://www.oscarblancarteblog.com/2018/06/25/creando-un-api-rest-en-java-parte-1/>
- <https://canigo.ctti.gencat.cat/canigo/framework/>
- <https://canigo.ctti.gencat.cat/canigo-documentacio-versions-3x/>
- <http://gencat.github.io/docs-canigo-v2/>
- https://es.wikipedia.org/wiki/Apache_Struts
- <http://www.jtech.ua.es/j2ee/publico/struts-2010-11/sesion01-struts-apuntes.html>
- http://www.ciberaula.com/articulo/introduccion_jstl
- <https://softwarecrafters.io/typescript/typescript-tutorial-javascript-introduccion>
- <https://www.typescriptlang.org/docs/home.html>
- <https://www.tutorialesprogramacionya.com/angularya/index.php?inicio=0>
- <https://desarrolloweb.com/articulos/introduccion-teorica-observables-angular.html>
- <https://codingpotions.com/angular-formularios>
- https://es.wikipedia.org/wiki/Integraci%C3%B3n_continua
- <https://aws.amazon.com/es/devops/continuous-integration/>
- <http://www.robertocrespo.net/kaizen/aprende-a-montar-un-entorno-de-integracion-continua-i/>



Avda. de Bruselas 35
28108 Alcobendas,
Madrid España
T +34 91 480 50 00
F +34 91 480 50 80
www.indracompany.com